pyAmpliCol 0.1.0

Methodology and Performance Report

Rikkert Frederix Valentin Hirschi Timea Vitos

Release 0.1.0

Abstract

This report defines the public theory, validation, and performance methodology for `pyAmpliCol`, release 0.1.0. It defines the canonical process coverage for the built-in Standard Model, the same physics loaded as a UFO/JSON Standard Model (UFO-SM), a scalar contact model, and a scalar-gravity model. Its tables present reproducible measurements tied to an effective configuration, execution environment, generated artifact, and independent-reference provenance.

Contents

1 Scope **2**

2 Current Recursion and Model Lowering **2**

3 Colour and Runtime Semantics **3**

4 Benchmark Methodology **3**

5 Standard-Model Process Matrices **5**

6 Z-plus-Jets Evaluator Ladders **28**

7 Colour-Singlet Model Ladders **31**

8 Interpretation **31**

9 Worked $d\bar{d} \to Zgg$ Example **32**

10 The $d\bar{d} \to Zgg$ DAG and Rusticol **35**

11 Reusable Leading-Colour Execution Layouts **39**

12 Prepared-Model Eager DAG Execution **41**

13 Arbitrary UFO and Serialized-JSON Models **45**

1 Scope

`pyAmpliCol` generates tree-level scattering-amplitude artifacts and evaluates them through the Rusticol runtime. The Python layer resolves models and processes and constructs a shared-current directed acyclic graph (DAG). In compiled mode, bounded stages become process-specific Symbolica evaluators. In eager mode, Python extracts proven fixed-width DAG columns and Rust lowers them to compact invocation tables over a prepared model-kernel pack. Both routes write a versioned process artifact. Rusticol loads that artifact and provides summed and resolved evaluation through one physics contract to Python, C11, C++17, Fortran 2008, and Rust 2021 clients.

The report has three purposes:

1. specify the observables and conditions required for a publishable benchmark;
2. preserve a stable matrix of representative process families and colour approximations; and
3. separate measured data from prose by rendering every result table from validated measurement records.

An N/A entry means that the current result set has no recorded measurement for that slot. It is neither a zero, a failure, nor a value carried over from another implementation. Diagnostic status labels, when present, identify a recorded campaign stop rather than a physics value. A cell marked SKIP records a deliberately held high-multiplicity measurement whose estimated or observed resource profile falls outside the campaign envelope.

2 Current Recursion and Model Lowering

2.1 Shared-current DAG

For an external subset I, particle state t, and local spin or flow label χ, an off-shell current has the schematic recursion

$$J^{\alpha}_{t,\chi}(I) = P^{\alpha}{}_{\beta}(t, p_I) \sum_{I=A\dot{\cup}B} V^{\beta}{}_{\gamma\delta}(t_A, t_B \to t) J^{\gamma}_{t_A,\chi_A}(A) J^{\delta}_{t_B,\chi_B}(B), \qquad p_I = \sum_{i\in I} p_i. \tag{1}$$

Current identity includes the particle, external labels, helicity ancestry, chirality or spin state, flavour and charge flow, colour state, momentum mask, and any auxiliary-current kind. Contributions with the same complete identity share one node. A topological sweep therefore evaluates each retained current once per phase-space point and reuses it in all dependent amplitudes.

External UFO or serialized JSON sources are compiled to the same canonical model representation as the process generator consumes. Particle metadata, interactions, coupling orders, Lorentz tensors, colour tensors, propagators, and runtime parameters come from that representation. UFO directories contain trusted executable Python; serialized JSON is the portable data-only source.

Optimizations on the external-model path are proof-gated rather than selected by particle names or PDG values. Exact normalized colour and Lorentz expressions, propagator metadata, and coupling relations certify current reflection, helicity, and reusable-kernel identities. For overlapping Standard-Model validation processes, the resulting current, interaction, amplitude-root, and colour-sector counts must match the built-in compatibility model after applying the same independently inferred perturbative-order envelope. An external compiler may retain fewer evaluator calls only when an additional exact kernel-equivalence certificate proves that reuse.

2.2 Contact terms and spin two

A vertex with valence above three is lowered to a deterministic tree of non-propagating auxiliary currents. The construction inserts the original coupling once and introduces no pole. Direct contraction of the original contact tensor is the validation oracle for the scalar-contact ladder.

The scalar-gravity ladder exercises rank-two external states and momentum-dependent interactions. For a massless graviton, the helicity-two polarization tensor is represented as

$$\epsilon^{\mu\nu}_{\pm 2} = \epsilon^{\mu}_{\pm}\epsilon^{\nu}_{\pm}. \tag{2}$$

Validation checks symmetry, transversality, tracelessness, normalization, and agreement between ordinary and higher-precision evaluation.

3 Colour and Runtime Semantics

The process matrices use three colour contracts:

LC	Leading-colour ordered amplitudes with physical colour-flow selectors.
NLC	The leading term plus the first retained $1/N_c^2$-suppressed contribution.
Full	The complete supported sparse colour metric for the generated amplitude basis.

For NLC, "retained" applies to a colour-matrix *entry*, not to a truncated numerical coefficient. Write the exact overlap of two colour-flow tensors as

$$C_{ab}(N_c) = \sum_p c_p^{(ab)} N_c^p, \qquad p_{ab} = \max\{p : c_p^{(ab)} \neq 0\}, \tag{3}$$

and let P be the leading power of the complete colour basis. The sparse NLC metric used by `build_color_contraction_plan()` in `src/pyamplicol/color/contraction.py` is

$$C_{ab}^{\mathrm{NLC}} = \begin{cases} C_{ab}(3), & p_{ab} \geq P-2, \\ 0, & p_{ab} < P-2. \end{cases} \tag{4}$$

Thus every qualifying entry keeps all of its finite-N_c terms. For three open fundamental lines and one adjoint, $P = 4$. Representative retained off-diagonal overlaps are

exact overlap	**highest power**	**NLC at $N_c = 3$**	**full at $N_c = 3$**
$-N_c^3 + N_c$	3	-24	-24
$N_c^2 - 1$	2	8	8

The implementation in `src/pyamplicol/color/contraction_factors.py` carries these powers as exact rational coefficients. The upper-triangular runtime plan applies a separate factor of two to off-diagonal interference terms; that storage factor must not be confused with the colour coefficient in Eq. (4).

The optimized `Runtime.evaluate()` path returns one summed matrix element per point.

The resolved method `Runtime.evaluate_resolved()` exposes LC values with axes `(point, helicity, flow)`. NLC and full-colour artifacts expose one already contracted colour component per helicity. Summing all non-point axes must reproduce the optimized total. Public selectors use physical helicity and flow identifiers; graph-local colour-sector identifiers are not report inputs.

All language bindings load the same process manifest and evaluator payloads. Cross-language agreement therefore validates the runtime interfaces and data layout. A separately generated reference is required for an independent physics comparison.

The default JIT artifact embeds a self-contained SymJIT application for f64 evaluation. Rusticol executes that payload through the separate MIT-licensed SymJIT runtime without importing Symbolica or consulting its runtime license state. Compiled ASM and C++ f64 payloads are likewise loaded directly by the native core. Python evaluation above f64 precision uses the retained Symbolica evaluator state and therefore requires Symbolica at runtime.

4 Benchmark Methodology

4.1 Configuration contract

Each measurement begins from an immutable resolved configuration. Unless a table variant says otherwise, the report contract uses the release defaults: requested SymJIT optimization level 3, evaluator batch size 128, output

chunk size 512, stage-local parameter layout, ten Horner iterations, backend-default common-pair elimination, a twenty-second benchmark target, two warmup runs, and at least five timed samples. The process matrices always request level 3; only explicitly labelled comparison rows, such as JIT level 1 in the dedicated Z ladder, override it. SymJIT may apply its documented backend fallback independently of the requested level. Requested and effective configurations are retained separately so resource or license adjustments remain visible.

Generation time is wall time for one explicit `Generator.generate()` operation starting from the resolved model input required by the selected execution mode and ending with a committed, loadable process artifact. It includes process expansion, DAG construction, validation-point preparation, artifact publication, and API-bundle emission. Compiled mode additionally includes process-stage evaluator construction and backend compilation. Eager mode instead includes proven-DAG column extraction, Rust plan lowering, and compact runtime-container serialization; preparation of its reusable model-kernel pack is excluded. Resolving the UFO/JSON or built-in model is performed before the process-generation timer and recorded separately as `model_precompile_seconds`. Eager prepared-pack creation is also a separate model-level measurement. Neither quantity is included in a process-generation cell. Existing artifacts may be benchmarked, but their generation slot remains N/A unless a matching generation measurement is available.

Runtime measurements use `BenchmarkRunner` and the summed `Runtime.evaluate()` observable. The reported wall time begins inside Rusticol after caller-language array conversion and covers repeated ordinary core evaluations, including source preparation, numerical execution, output assignment, selector work, and reduction. The secondary execution submetric is mode-specific: in compiled mode it is the time inside generated stage and amplitude evaluators; in eager mode it is the inclusive eager execution lane, which contains gather, prepared-kernel calls, scatter, finalization, closure, and eager reduction. Python/NumPy packing and language-binding conversion are therefore excluded from both reported quantities. A measurement records sample count, time per point, uncertainty, effective settings, software revision, artifact identity, hardware, and execution environment. Comparisons are valid only for the same process, colour accuracy, parameters, phase-space ensemble, precision, selectors, and normalization.

4.2 Benchmark platform

The populated measurements in this report were produced on the cluster host `itphlies.clan` from the Nix developer shell, not from a scheduler-visible batch partition in the reporting environment. The node reports Linux `6.12.63` on NixOS `26.05`, `x86_64`, glibc `2.42`. The processor is a dual-socket AMD EPYC 9754 system (Genoa/Zen 4): 256 physical cores are visible across eight NUMA nodes, with 384 logical CPUs online in this session. The advertised instruction-set families include SSE through SSE4.2, FMA, AVX, AVX2, AVX-512 foundation/vector, VNNI/BF16, AES/SHA, BMI1/2, ADX, VAES, and carry-less multiplication. Installed memory reported by the kernel is `1188419660 kB` ($\simeq$ 1.11 TiB), with no configured swap.

All report measurements use the repository's developer installation through `.venv/bin/python` inside `nix develop`. The recorded toolchain is Python `3.11.15` with NumPy `2.4.2`, Rust `1.89.0`, Cargo `1.89.0`, GCC/G++/GNU Fortran `15.2.0`, and pdfTeX `3.141592653-2.6-1.40.27` from TeX Live 2025. Unless a cell's manifest states otherwise, generation and evaluation use one core and runtime sampling targets twenty seconds. Resource ceilings and timeouts are retained in each measurement record; a limit reached is rendered as a status rather than a timing value. Collection concurrency and scheduler queue time are not part of a cell's workload. Wall times are native Rusticol repeated-core timings after caller-language momentum packing, and model compilation is recorded separately from process-generation time.

4.3 Numerical validation

The matrix element and its independent reference are evaluated at the same deterministic points with the same parameter card. Shared-core language checks use relative tolerance 10^{-12} and absolute tolerance 10^{-15}. Independent reference comparisons use relative tolerance 10^{-8}, with higher precision used to diagnose unstable points. A populated result must also satisfy the resolved-sum identity and preserve expected structural zeros.

No timing is accepted as a numerical validation result. Conversely, a valid matrix element does not populate generation or runtime fields without a matching timing run. A result enters the report only after its record is internally consistent and its pointwise comparison has the status required by the table.

5 Standard-Model Process Matrices

The canonical matrix spans electroweak vector production, charged-current and leptonic channels, heavy-quark production, pure gluons, multi-boson final states, Higgs-associated final states, and three- and four-quark-line cases. Here n is the number of final-state particles. LC coverage retains the declared grid through $n = 9$, with the four bounded families ending at $n = 8$. NLC and full-colour grids retain $n = 1, \ldots, 5$; cells below a family's minimum physical multiplicity remain explicitly marked N/A.

The UFO-SM matrices use the same process definitions, colour contracts, and numerical input parameters as the built-in-SM matrices. Their source is the packaged Standard Model loaded through the generic UFO/JSON compiler. The comparison therefore isolates model loading and generic lowering without changing the requested physics or the original Fortran `AmpliCol` reference.

How to read a process-matrix cell

Every process-matrix comparison uses `pyAmpliCol` JIT at requested optimization level 3. The process matrices do not include ASM or C++ measurements; those backends appear in the dedicated Z ladder. For a time T, a parenthesized `x`r is the dimensionless ratio

$$r = \frac{T_{\texttt{pyAmpliCol}}}{T_{\texttt{AmpliCol}}}. \tag{5}$$

Green, orange, and red denote respectively $r < 1$, $1 \leq r < 2$, and $r \geq 2$; thus green means that `pyAmpliCol` is faster. The compact layout is:

contract	line and reference entry	pyAmpliCol entries
LC, upper	`AmpliCol` generation time in seconds	selected-flow/helicity-sum generation ratio; all-flows/fixed-helicity generation ratio
LC, lower	`AmpliCol` selected-flow/all-flows runtimes in microseconds per point, separated by "/"	for each workload, `(xwall|eval)` gives the Rusticol-core wall and evaluator-only ratios against the matching `AmpliCol` runtime
NLC/full, upper	`AmpliCol` generation time in seconds	`pyAmpliCol` JIT O3 generation ratio
NLC/full, lower	`AmpliCol` fully contracted runtime in microseconds per point	`(xwall|eval)` gives the Rusticol-core wall and evaluator-only ratios against that runtime

The two LC generation ratios correspond to separately generated, complete-coverage `pyAmpliCol` artifacts with the layouts defined in Sec. 11. The topology-replay artifact supplies the selected-flow/helicity-sum workload; the all-flow-union artifact supplies the all-flows/fixed-helicity workload. Both retain every physical LC flow and helicity, and the displayed flow or helicity is selected at runtime. `AmpliCol` generates one library that supports both workloads, so the same `AmpliCol` generation time is the denominator of both ratios. "Selected flow, helicity sum" evaluates one matched physical LC ordering and sums all supported helicities. "All flows, fixed helicity" evaluates every represented physical LC ordering for one supported helicity. NLC and full colour instead report one helicity-summed, fully colour-contracted workload; no per-flow scalar is defined for those contracts. Exception: for the four-open-quark-line LC family the `AmpliCol` colour probe has no reference value, so `AmpliCol` slots are N/A, `pyAmpliCol` entries are absolute selected/all-flow times, and the row is omitted from multiplier summaries.

At the foot of each multiplicity column, every summary cell has two lines. The first contains the `AmpliCol` values as `min|max|mean|median`; generation is in seconds and runtime in microseconds per point. The second contains the paired `pyAmpliCol/ AmpliCol` multipliers as `min|max|mean|median|ratio-of-sums`,

where the last field is $\sum_i T_{\texttt{pyAmpliCol},i} / \sum_i T_{\texttt{AmpliCol},i}$ over the same valid process pairs. LC provides separate generation and runtime summary rows for its two workloads. NLC and full colour provide generation, core-wall runtime, and evaluator-only runtime rows; the last two share the same `AmpliCol` reference runtime because `AmpliCol` has no corresponding evaluator-only timing split.

Compiled-versus-eager UFO-SM matrices

The three additional UFO-SM matrices retain exactly the same process grid, colour contracts, typography, pagination, and cell geometry, but change the comparison pair. The reference is the existing compiled JIT O3 measurement; the candidate is eager-DAG JIT O3. No `AmpliCol` measurement is repeated or used in these pages. For a timing quantity T, their displayed multiplier is therefore

$$r_{\text{eager/compiled}} = \frac{T_{\text{eager-DAG JIT O3}}}{T_{\text{compiled JIT O3}}}. \tag{6}$$

The same green, orange, and red thresholds apply, so green now means that eager-DAG execution or generation is faster than compiled execution or generation.

In an LC eager cell, the upper-left reference slot contains the two compiled generation times as `replay/union`. Compiled and eager modes each generate one complete topology-replay artifact and one complete all-flow-union artifact. Candidate generation ratios are layout-matched: eager replay over compiled replay, followed by eager union over compiled union. The lower-left slot retains the two absolute compiled Rusticol-core wall times. Each candidate `(xwall|eval)` pair is the eager core-wall and inclusive eager-execution time divided by the matching compiled core-wall time. NLC and full colour use the same four-field layout as before, with one compiled generation/runtime reference and one eager generation plus wall/evaluator ratio pair.

The eager process-generation timer starts from an already prepared UFO-SM model bundle. Preparing its reusable JIT O3 canonical-kernel pack is an architecture-specific, model-level operation: the report records that setup cost and its ABI provenance separately, and excludes it from every process cell. Pointwise validation uses the same deterministic phase-space point, parameters, colour contract, and selectors as the joined compiled entry. For LC, both the selected-flow/helicity-sum and all-flows/fixed-helicity observables must agree; NLC and full colour compare the contracted total.

5.1 Built-in SM LC process matrix

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	7.81 0.114 / 0.128	(x0.035) (x3.98|0.499)	(x0.0248) (x1.56|0.219)	7.73 0.454 / 0.361	(x0.0402) (x1.95|0.462)	(x0.0298) (x0.983|0.195)	7.43 1.21 / 0.931	(x0.0924) (x1.46|0.54)	(x0.0501) (x0.746|0.225)
2	$u\bar{d} \to W^+ + (n-1)g$	7.51 0.0658 / 0.131	(x0.0274) (x5.15|0.608)	(x0.0219) (x1.51|0.191)	7.73 0.305 / 0.287	(x0.0334) (x2.18|0.443)	(x0.023) (x1.23|0.236)	7.47 0.7 / 0.826	(x0.0716) (x1.74|0.545)	(x0.0366) (x0.753|0.234)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			9.68 0.265 / 0.331	(x0.0411) (x2.17|0.406)	(x0.0294) (x0.964|0.202)	9.31 0.626 / 0.5	(x0.0522) (x1.48|0.389)	(x0.0389) (x0.925|0.245)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			9.29 0.116 / 0.212	(x0.0275) (x2.91|0.442)	(x0.018) (x1.42|0.246)	9.12 0.227 / 0.417	(x0.0314) (x2.53|0.524)	(x0.0212) (x1.07|0.232)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			9.76 0.554 / 0.383	(x0.0328) (x1.91|0.481)	(x0.0244) (x0.935|0.195)	9.49 2.07 / 0.671	(x0.0649) (x1.08|0.457)	(x0.0346) (x0.825|0.23)
6	$gg \to t\bar{t} + (n-2)g$	N/A			9.41 0.407 / 0.429	(x0.0385) (x2.55|0.652)	(x0.0216) (x1.05|0.284)	17.3 1.93 / 1.67	(x0.0964) (x1.29|0.583)	(x0.032) (x0.739|0.36)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			9.12 0.232 / 0.371	(x0.04) (x2.95|0.504)	(x0.0263) (x1.1|0.258)	12.9 0.712 / 1.02	(x0.0789) (x2.12|0.766)	(x0.0397) (x0.868|0.329)
8	$gg \to gg + (n-2)g$	N/A			11.6 0.517 / 0.71	(x0.0442) (x1.59|0.475)	(x0.0216) (x0.767|0.245)	12.6 2.37 / 3.82	(x0.167) (x0.987|0.532)	(x0.0642) (x0.526|0.305)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			18 4.65 / 0.878	(x0.0636) (x0.817|0.432)	(x0.0241) (x0.734|0.257)
10	$d\bar{d} \to e^+e^- ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

gen O3 one-flow, hel. sum	7.51| x0.0274|	7.81| x0.035|	7.66| x0.0312|	7.66 x0.0312|	x0.0313	7.73| x0.0275|	11.6| x0.0442|	9.29| x0.0372|	9.35 x0.0392|	x0.0374	7.43| x0.0314|	18| x0.167|	11.5| x0.0798|	9.49 x0.0716|	x0.0825
gen O3 all-flows, one-hel	7.51| x0.0219|	7.81| x0.0248|	7.66| x0.0233|	7.66 x0.0233|	x0.0233	7.73| x0.018|	11.6| x0.0298|	9.29| x0.0243|	9.35 x0.0237|	x0.0241	7.43| x0.0212|	18| x0.0642|	11.5| x0.0379|	9.49 x0.0366|	x0.037
run O3 one-flow, hel. sum	0.0658| x3.98|	0.114| x5.15|	0.0897| x4.56|	0.0897 x4.56|	x4.4	0.116| x1.59|	0.554| x2.95|	0.356| x2.28|	0.356 x2.17|	x2.13	0.227| x0.817|	4.65| x2.53|	1.61| x1.5|	1.21 x1.46|	x1.16
run O3 all-flows, one-hel	0.128| x1.51|	0.131| x1.56|	0.129| x1.54|	0.129 x1.54|	x1.54	0.212| x0.767|	0.71| x1.42|	0.385| x1.06|	0.366 x1.01|	x1	0.417| x0.526|	3.82| x1.07|	1.19| x0.798|	0.878 x0.753|	x0.703

Built-in SM LC process matrix (continued)

ID	base process	n=4			n=5			n=6		
1	$d\bar{d} \to Z + (n-1)g$	7.73 4.2 / 3.02	(x0.25) (x0.935|0.472)	(x0.152) (x0.578|0.252)	7.92 12.3 / 15.5	(x1.25) (x0.755|0.449)	(x0.829) (x0.442|0.254)	8.65 34.2 / 97.2	(x11.1) (x0.782|0.495)	(x8.02) (x0.698|0.345)
2	$u\bar{d} \to W^+ + (n-1)g$	7.7 2.58 / 3.06	(x0.168) (x1.08|0.52)	(x0.0907) (x0.566|0.253)	7.77 8.28 / 15.7	(x0.799) (x0.806|0.478)	(x0.514) (x0.408|0.248)	7.91 23.9 / 96.5	(x7.72) (x0.744|0.481)	(x5.42) (x0.685|0.347)
3	$d\bar{d} \to e^+e^- + (n-2)g$	9.64 1.33 / 0.976	(x0.107) (x1.25|0.471)	(x0.0652) (x0.874|0.301)	9.75 3.95 / 3.53	(x0.273) (x0.928|0.474)	(x0.18) (x0.571|0.264)	10.8 11.1 / 16.7	(x1.17) (x0.767|0.463)	(x1.42) (x0.471|0.27)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.36 0.498 / 0.903	(x0.0542) (x1.99|0.607)	(x0.0298) (x0.814|0.27)	9.5 1.75 / 3.11	(x0.127) (x1.28|0.582)	(x0.0729) (x0.576|0.269)	9.74 6.02 / 15.2	(x0.537) (x1.08|0.603)	(x0.354) (x0.599|0.353)
5	$d\bar{d} \to ZZ + (n-2)g$	10.1 6.36 / 1.7	(x0.183) (x0.78|0.397)	(x0.0642) (x0.599|0.227)	10.7 16.7 / 6.32	(x0.557) (x0.684|0.399)	(x4.53) (x0.716|0.395)	12.9 43.2 / 30.8	(x2.81) (x0.852|0.478)	(x1.5) (x0.374|0.211)
6	$gg \to t\bar{t} + (n-2)g$	32.5 6.91 / 8.59	(x0.32) (x0.912|0.512)	(x0.0846) (x0.603|0.396)	63.1 20.7 / 55.2	(x1.29) (x0.881|0.531)	(x0.465) (x0.967|0.531)	180 58 / 487	(x1.16) (x1.23|0.661)	(x2.04) (x1.14|0.541)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	17.9 2.44 / 4.18	(x0.31) (x1.71|0.932)	(x0.0893) (x0.672|0.352)	23.7 8.13 / 23.9	(x1.22) (x5|3.08)	(x0.85) (x0.515|0.316)	32.7 23.7 / 160	(x2.31) (x1.93|1.21)	(x3.03) (x0.969|0.46)
8	$gg \to gg + (n-2)g$	15.7 9.3 / 26.9	(x0.881) (x0.694|0.46)	(x0.407) (x0.429|0.3)	19.1 28.7 / 219	(x1.42) (x0.611|0.442)	(x4.49) (x0.781|0.444)	26.5 79.3 / 6.05e3	(x4.1) (x0.903|0.606)	(x42.9) (x0.393|0.186)
9	$d\bar{d} \to ZZZ + (n-3)g$	20.1 12.9 / 1.57	(x0.133) (x0.663|0.405)	(x0.0295) (x0.658|0.249)	25.6 33.2 / 3.92	(x0.347) (x0.722|0.43)	(x0.0628) (x1.88|0.803)	37.4 83.7 / 14.5	(x0.745) (x1.31|0.561)	(x0.262) (x0.377|0.19)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	57.3 2.15 / 1.3	(x0.0221) (x1.29|0.713)	(x0.013) (x0.766|0.309)	61.2 4.67 / 1.95	(x0.0319) (x3.59|2.03)	(x0.0383) (x1.17|0.429)	65.6 10.7 / 3.88	(x0.0655) (x0.818|0.521)	(x0.0263) (x0.818|0.277)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	31.9 3.02 / 1.66	(x0.0574) (x1.1|0.6)	(x0.0212) (x0.909|0.414)	63 9.04 / 4.76	(x0.89) (x1.6|0.908)	(x0.0267) (x0.771|0.417)	107 26.1 / 21.2	(x0.519) (x1.29|0.825)	(x0.0747) (x0.529|0.326)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	58.6 3.75 / 1.8	(x0.0375) (x0.704|0.391)	(x0.0283) (x0.612|0.26)	61.3 7.84 / 2.65	(x0.0518) (x0.588|0.35)	(x0.0378) (x2.12|0.902)	66 17.1 / 4.92	(x0.108) (x0.639|0.459)	(x0.0621) (x0.548|0.258)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	11.8 0.538 / 2.14	(x0.15) (x2.21|0.721)	(x0.11) (x0.761|0.329)	20 1.22 / 9.32	(x0.515) (x6.54|1.69)	(x0.24) (x0.601|0.322)	32.5 3.02 / 54	(x2.63) (x1.59|0.594)	(x1.01) (x0.824|0.391)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A N/A / N/A	37.2 s 6.41 us|3.71 us	23.1 s 18.2 us|10.6 us

gen O3 one-flow, hel. sum	7.7|	58.6|	22.3|	15.7		7.77|	63.1|	29.4|	20		7.91|	180|	46|	32.5	
	x0.0221|	x0.881|	x0.206|	x0.15|	x0.159	x0.0319|	x1.42|	x0.675|	x0.557|	x0.637	x0.0655|	x11.1|	x2.69|	x1.17|	x1.31
gen O3 all-flows, one-hel	7.7|	58.6|	22.3|	15.7		7.77|	63.1|	29.4|	20		7.91|	180|	46|	32.5	
	x0.013|	x0.407|	x0.0911|	x0.0652|	x0.0659	x0.0267|	x4.53|	x0.95|	x0.24|	x0.548	x0.0263|	x42.9|	x5.09|	x1.42|	x3.03
run O3 one-flow, hel. sum	0.498|	12.9|	4.3|	3.02		1.22|	33.2|	12|	8.28		3.02|	83.7|	32.3|	23.9	
	x0.663|	x2.21|	x1.18|	x1.08|	x0.889	x0.588|	x6.54|	x1.85|	x0.881|	x1.13	x0.639|	x1.93|	x1.07|	x0.903|	x1.08
run O3 all-flows, one-hel	0.903|	26.9|	4.44|	1.8		1.95|	219|	28.1|	6.32		3.88|	6.05e3|	542|	30.8	
	x0.429|	x0.909|	x0.68|	x0.658|	x0.552	x0.408|	x2.12|	x0.886|	x0.716|	x0.775	x0.374|	x1.14|	x0.648|	x0.599|	x0.47

Built-in SM LC process matrix (continued)

ID	base process	n=7			n=8			n=9		
1	$d\bar{d} \to Z + (n-1)g$	10.2 91.2 , 858	(x2.32) (x1.01|0.589)	(x78.4) (x0.767|0.31)	18.2 229 , 2.15e4	(N/A) N/A	(N/A) N/A	43 575 , 2.61e5	(N/A) N/A	(N/A) N/A
2	$u\bar{d} \to W^+ + (n-1)g$	8.89 65.5 / 965	(x2.06) (x0.922|0.605)	(x60.6) (x0.724|0.274)	11.5 172 / 2.53e4	(N/A) N/A	(N/A) N/A	18.7 430 / 2.3e5	(N/A) N/A	(N/A) N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	11.1 31.4 , 105	(x9.16) (x0.66|0.436)	(x8.01) (x0.643|0.325)	13 82.4 , 873	(x1.86) (x0.935|0.621)	(x74.5) (x1.12|0.361)	19.5 206 , 1.56e4	(N/A) N/A	(N/A) N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	10.2 19.1 / 96.3	(x4.22) (x0.727|0.487)	(x3.22) (x0.62|0.342)	10.7 53.5 / 833	(x1.51) (x0.806|0.553)	(x36) (x0.717|0.301)	12.6 142 / 1.88e4	(N/A) N/A	(N/A) N/A
5	$d\bar{d} \to ZZ + (n-2)g$	20 110 , 192	(x14.6) (x0.974|0.535)	(x9.62) (x0.692|0.268)	43.2 266 , 2.22e3	(x1.45) (x0.873|0.52)	(x51.9) (x0.433|0.172)	120 661 , 3.97e4	(N/A) N/A	(N/A) N/A
6	$gg \to t\bar{t} + (n-2)g$	971 159 / 9.75e3	(x0.749) (x1.11|0.601)	(x4.96) (x0.53|0.226)	1.52e4 390 / 1.69e5	(N/A) N/A	(N/A) N/A	N/A		
7	$d\bar{d} \to t\bar{t} + (n-2)g$	61.2 64.9 , 1.31e3	(x4.32) (x1.93|1.22)	(x17.9) (x0.851|0.378)	408 170 , 2.55e4	(N/A) N/A	(N/A) N/A	N/A		
8	$gg \to gg + (n-2)g$	54.6 211 / 6.29e4	(N/A) N/A	(N/A) N/A	N/A			N/A		
9	$d\bar{d} \to ZZZ + (n-3)g$	88.9 204 , 66.9	(x1.47) (x0.868|0.502)	(x0.768) (x0.773|0.434)	251 476 , 417	(x4.66) (x0.896|0.511)	(x3.02) (x0.521|0.21)	743 1.44e3 , 7.45e3	(N/A) N/A	(N/A) N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	77.1 25.6 / 12.5	(x0.162) (x0.774|0.508)	(x0.0775) (x0.453|0.231)	104 61 / 55.9	(x0.792) (x0.942|0.616)	(x0.361) (x0.367|0.204)	155 144 / 321	(x2.83) (x1.04|0.605)	(x2.1) (x0.643|0.262)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	235 71.6 , 109	(x1.12) (x2.26|1.21)	(x0.269) (x0.91|0.408)	775 187 , 723	(x0.8) (x1.91|1.17)	(x0.729) (x0.926|0.347)	1.31e4 465 , 1.61e4	(N/A) N/A	(N/A) N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	78.8 39.2 / 14.3	(x0.243) (x0.481|0.311)	(x0.155) (x0.417|0.225)	108 92 / 62.3	(x1.07) (x0.674|0.401)	(x0.772) (x0.567|0.278)	171 209 / 370	(x4.04) (x0.635|0.394)	(x3.32) (x0.575|0.245)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	57.5 8.11 , 364	(x4.98) (x1.43|0.512)	(x4.58) (x0.963|0.407)	115 20.8 , 3.82e3	(N/A) N/A	(N/A) N/A	N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

gen O3 one-flow, hel. sum	8.89|	971|	130|	57.5		10.7|	1.52e4|	1.42e3|	106		12.6|	1.31e4|	1.59e3|	120	
	x0.162|	x14.6|	x3.78|	x2.19|	x1.34	x0.792|	x4.66|	x1.73|	x1.45|	x1.6	x2.83|	x4.04|	x3.43|	x3.43|	x3.46
gen O3 all-flows, one-hel	8.89|	971|	136|	59.3		10.7|	775|	186|	104		155|	171|	163|	163	
	x0.0775|	x78.4|	x15.7|	x4.77|	x4.89	x0.361|	x74.5|	x23.9|	x3.02|	x3.86	x2.1|	x3.32|	x2.71|	x2.71|	x2.74
run O3 one-flow, hel. sum	8.11|	211|	84.7|	65.5		20.8|	476|	183|	171		142|	1.44e3|	475|	430	
	x0.481|	x2.26|	x1.1|	x0.948|	x1.11	x0.674|	x1.91|	x1.01|	x0.896|	x1.03	x0.635|	x1.04|	x0.836|	x0.836|	x0.799
run O3 all-flows, one-hel	12.5|	6.29e4|	5.9e3|	192		55.9|	1.69e5|	2.09e4|	1.55e3		321|	2.61e5|	6.55e4|	1.61e4	
	x0.417|	x0.963|	x0.695|	x0.708|	x0.608	x0.367|	x1.12|	x0.665|	x0.567|	x0.671	x0.575|	x0.643|	x0.609|	x0.609|	x0.607

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.2 Built-in SM NLC process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	7.84 s 0.227 us	(x0.0394) (x1.77|0.234)	7.4 s 0.661 us	(x0.0483) (x1.23|0.297)	6.91 s 3.71 us	(x0.0925) (x0.639|0.293)
2	$u\bar{d} \to W^+ + (n-1)g$	7.35 s 0.217 us	(x0.0366) (x1.73|0.177)	7.11 s 0.375 us	(x0.0451) (x1.85|0.363)	6.82 s 1.98 us	(x0.071) (x0.851|0.337)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		9.32 s 0.556 us	(x0.0478) (x0.942|0.188)	8.42 s 1.15 us	(x0.0803) (x0.817|0.231)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		9.21 s 0.246 us	(x0.0324) (x1.8|0.204)	8.47 s 0.548 us	(x0.0453) (x1.32|0.228)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		9.47 s 1.03 us	(x0.0386) (x0.929|0.241)	8.47 s 3.04 us	(x0.0701) (x0.712|0.32)
6	$gg \to t\bar{t} + (n-2)g$	N/A		9.19 s 1.67 us	(x0.0425) (x0.799|0.274)	14.9 s 16.3 us	(x0.0919) (x0.545|0.287)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		9.27 s 1.24 us	(x0.042) (x1.57|0.199)	11.9 s 6.14 us	(x0.0888) (x1.16|0.329)
8	$gg \to gg + (n-2)g$	N/A		11.4 s 8 us	(x0.0491) (x0.285|0.149)	13.7 s 121 us	(x0.389) (x0.395|0.18)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		17.8 s 5.69 us	(x0.0482) (x0.768|0.351)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		7.35| 7.84| 7.6| 7.6 (x0.0366)|(x0.0394)| (x0.038)| (x0.038)|(x0.0381)		7.11| 11.4| 9.04| 9.24 (x0.0324)|(x0.0491)|(x0.0432)|(x0.0438)|(x0.0432)		6.82| 17.8| 10.8| 8.47 (x0.0453)| (x0.389)| (x0.109)|(x0.0803)| (x0.117)	
summary: run wall		0.217| 0.227| 0.222| 0.222 (x1.73)| (x1.77)| (x1.75)| (x1.75)| (x1.75)		0.246| 8| 1.72| 0.846 (x0.285)| (x1.85)| (x1.18)| (x1.09)| (x0.653)		0.548| 121| 17.7| 3.71 (x0.395)| (x1.32)| (x0.801)| (x0.768)| (x0.477)	
summary: run eval		0.217| 0.227| 0.222| 0.222 (x0.177)| (x0.234)| (x0.206)| (x0.206)| (x0.206)		0.246| 8| 1.72| 0.846 (x0.149)| (x0.363)| (x0.239)| (x0.222)| (x0.191)		0.548| 121| 17.7| 3.71 (x0.18)| (x0.351)| (x0.284)| (x0.293)| (x0.211)	

Built-in SM NLC process matrix (continued)

ID	base process	n=4		n=5	
1	$d\bar{d} \to Z + (n-1)g$	7.7 s 31.2 us	(x0.378) (x0.647|0.326)	8.07 s 364 us	(x3.25) (x0.891|0.341)
2	$u\bar{d} \to W^+ + (n-1)g$	7.39 s 18.3 us	(x0.248) (x0.549|0.315)	8.18 s 233 us	(x2.08) (x0.832|0.33)
3	$d\bar{d} \to e^+e^- + (n-2)g$	9.67 s 5.45 us	(x0.0871) (x0.406|0.184)	10 s 44.9 us	(x0.183) (x0.288|0.218)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.6 s 2.55 us	(x0.0471) (x0.556|0.201)	9.67 s 22.4 us	(x0.0873) (x0.302|0.181)
5	$d\bar{d} \to ZZ + (n-2)g$	9.86 s 15.4 us	(x0.151) (x0.49|0.295)	10.8 s 117 us	(x0.784) (x0.759|0.381)
6	$gg \to t\bar{t} + (n-2)g$	32.6 s 217 us	(x0.517) (x0.778|0.315)	62.5 s 3.47e3 us	(x3.35) (x0.837|0.234)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	18.8 s 56.7 us	(x0.34) (x1.27|0.401)	28.3 s 691 us	(x1.96) (x1.15|0.38)
8	$gg \to gg + (n-2)g$	17 s 2.02e3 us	(x5.52) (x0.42|0.171)	22.9 s 3.64e4 us	(x54.7) (x0.511|0.164)
9	$d\bar{d} \to ZZZ + (n-3)g$	21 s 15.1 us	(x0.0859) (x0.53|0.355)	24.9 s 74.5 us	(x0.22) (x0.682|0.393)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	61.7 s 3.9 us	(x0.0173) (x0.699|0.391)	64.8 s 8.79 us	(x0.00951) (x0.498|0.333)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	34.9 s 11.7 us	(x0.0462) (x0.697|0.358)	68.2 s 64.7 us	(x0.081) (x0.865|0.391)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	62.7 s 7.33 us	(x0.0315) (x0.355|0.202)	67.5 s 15.5 us	(x0.00857) (x0.265|0.18)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	12.1 s 143 us	(x0.146) (x0.0521|0.014)	23.8 s 1.29e3 us	(x0.189) (x0.0501|0.0162)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A	
	summary: gen	7.39| 62.7| 23.5| 17 (x0.0173)| (x5.52)| (x0.585)| (x0.146)| (x0.435)		8.07| 68.2| 31.5| 23.8 (x0.00857)| (x54.7)| (x5.15)| (x0.22)| (x3.88)	
	summary: run wall	2.55| 2.02e3| 196| 15.4 (x0.0521)| (x1.27)| (x0.573)| (x0.549)| (x0.455)		8.79| 3.64e4| 3.29e3| 117 (x0.0501)| (x1.15)| (x0.61)| (x0.682)| (x0.54)	
	summary: run eval	2.55| 2.02e3| 196| 15.4 (x0.014)| (x0.401)| (x0.271)| (x0.315)| (x0.186)		8.79| 3.64e4| 3.29e3| 117 (x0.0162)| (x0.393)| (x0.272)| (x0.33)| (x0.172)	

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.3 Built-in SM full-colour process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	7.26 s 0.272 us	(x0.0425) (x1.48|0.197)	7.64 s 0.813 us	(x0.0483) (x1.02|0.263)	7.84 s 4.02 us	(x0.0793) (x0.745|0.271)
2	$u\bar{d} \to W^+ + (n-1)g$	7.61 s 0.23 us	(x0.0352) (x1.61|0.167)	7.3 s 0.5 us	(x0.0425) (x1.4|0.279)	7.58 s 2.18 us	(x0.0702) (x0.779|0.315)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		9.2 s 0.612 us	(x0.0453) (x0.861|0.172)	9.66 s 1.27 us	(x0.0546) (x0.701|0.199)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		9.07 s 0.442 us	(x0.0343) (x0.998|0.112)	9.45 s 0.636 us	(x0.036) (x1.1|0.191)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		9.39 s 1.13 us	(x0.0391) (x0.854|0.237)	9.53 s 3.26 us	(x0.0602) (x0.627|0.283)
6	$gg \to t\bar{t} + (n-2)g$	N/A		9.37 s 2.04 us	(x0.0423) (x0.66|0.231)	17.4 s 19.5 us	(x0.0798) (x0.505|0.244)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		9.27 s 1.52 us	(x0.0418) (x1.29|0.162)	13.1 s 7.42 us	(x0.0707) (x0.95|0.269)
8	$gg \to gg + (n-2)g$	N/A		10.9 s 9.34 us	(x0.0515) (x0.244|0.128)	13.9 s 148 us	(x0.386) (x0.357|0.144)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		18.2 s 6.13 us	(x0.0476) (x0.571|0.328)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		7.26| (x0.0352)	7.61| (x0.0425) | 7.43| (x0.0389) | 7.43 (x0.0389) | (x0.0388)	7.3| (x0.0343)	10.9| (x0.0515) | 9.01| (x0.0432) | 9.23 (x0.0424) | (x0.0433)	7.58| (x0.036)	18.2| (x0.386) | 11.8| (x0.0983) | 9.66 (x0.0702) | (x0.104)
summary: run wall		0.23| (x1.48)	0.272| (x1.61) | 0.251| (x1.55) | 0.251 (x1.55) | (x1.54)	0.442| (x0.244)	9.34| (x1.4) | 2.05| (x0.917) | 0.971 (x0.93) | (x0.552)	0.636| (x0.357)	148| (x1.1) | 21.3| (x0.703) | 4.02 (x0.701) | (x0.424)
summary: run eval		0.23| (x0.167)	0.272| (x0.197) | 0.251| (x0.182) | 0.251 (x0.182) | (x0.183)	0.442| (x0.112)	9.34| (x0.279) | 2.05| (x0.198) | 0.971 (x0.201) | (x0.164)	0.636| (x0.144)	148| (x0.328) | 21.3| (x0.25) | 4.02 (x0.269) | (x0.172)

Built-in SM full-colour process matrix (continued)

ID	base process	n=4		n=5	
1	$d\bar{d} \to Z + (n-1)g$	7.36 s 35.9 us	(x0.393) (x0.581|0.28)	8.23 s 464 us	(x3.15) (x0.794|0.251)
2	$u\bar{d} \to W^+ + (n-1)g$	7.78 s 20.6 us	(x0.228) (x0.513|0.278)	7.71 s 281 us	(x2.25) (x0.82|0.286)
3	$d\bar{d} \to e^+e^- + (n-2)g$	10 s 6.06 us	(x0.0809) (x0.362|0.167)	10.3 s 50.7 us	(x0.184) (x0.275|0.157)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.58 s 2.75 us	(x0.0485) (x0.521|0.185)	9.73 s 25.4 us	(x0.0851) (x0.264|0.16)
5	$d\bar{d} \to ZZ + (n-2)g$	10.1 s 16.7 us	(x0.161) (x0.473|0.282)	11.1 s 131 us	(x0.76) (x0.696|0.341)
6	$gg \to t\bar{t} + (n-2)g$	29.4 s 284 us	(x0.574) (x0.746|0.244)	62.8 s 6.01e3 us	(x3.69) (x1.03|0.14)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	18.9 s 73.5 us	(x0.338) (x1.04|0.31)	28.3 s 1.09e3 us	(x2.03) (x0.886|0.248)
8	$gg \to gg + (n-2)g$	17 s 3.07e3 us	(x5.86) (x0.476|0.119)	22.8 s 9.53e4 us	(x85.5) (x0.832|0.0675)
9	$d\bar{d} \to ZZZ + (n-3)g$	21.9 s 16 us	(x0.0791) (x0.5|0.334)	25.7 s 78.8 us	(x0.215) (x0.647|0.368)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	59.8 s 4.42 us	(x0.0196) (x0.618|0.345)	63.6 s 9.14 us	(x0.00967) (x0.479|0.318)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	34.6 s 12.6 us	(x0.0474) (x0.831|0.333)	67.3 s 68.7 us	(x0.0819) (x0.822|0.369)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	69.6 s 7.6 us	(x0.0294) (x0.341|0.195)	66.7 s 16 us	(x0.00902) (x0.265|0.18)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	12.7 s 147 us	(x0.158) (x0.0507|0.0136)	23.8 s 1.36e3 us	(x0.19) (x0.0484|0.0153)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A	
	summary: gen	7.36| 69.6| 23.7| 17 (x0.0196)| (x5.86)| (x0.617)| (x0.158)| (x0.45)		7.71| 67.3| 31.4| 23.8 (x0.00902)| (x85.5)| (x7.55)| (x0.215)| (x5.66)	
	summary: run wall	2.75| 3.07e3| 285| 16.7 (x0.0507)| (x1.04)| (x0.543)| (x0.513)| (x0.494)		9.14| 9.53e4| 8.07e3| 131 (x0.0484)| (x1.03)| (x0.604)| (x0.696)| (x0.832)	
	summary: run eval	2.75| 3.07e3| 285| 16.7 (x0.0136)| (x0.345)| (x0.237)| (x0.278)| (x0.134)		9.14| 9.53e4| 8.07e3| 131 (x0.0153)| (x0.369)| (x0.223)| (x0.248)|(x0.0751)	

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.4 UFO-SM LC process matrix

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	7.68 0.126 / 0.202	(x0.0489) (x3.6|0.479)	(x0.0401) (x1.03|0.161)	7.49 0.452 / 0.361	(x0.0606) (x2|0.512)	(x0.0481) (x1.03|0.223)	7.49 1.28 / 0.911	(x0.12) (x1.43|0.562)	(x0.0642) (x0.736|0.235)
2	$u\bar{d} \to W^+ + (n-1)g$	7.64 0.0319 / 0.203	(x0.0417) (x10.9|1.27)	(x0.0359) (x1.05|0.122)	7.63 0.305 / 0.286	(x0.0485) (x2.31|0.471)	(x0.0408) (x1.29|0.259)	7.44 0.785 / 0.831	(x0.0942) (x1.6|0.517)	(x0.0509) (x0.787|0.25)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			9.58 0.266 / 0.333	(x0.0627) (x2.16|0.418)	(x0.0481) (x0.979|0.218)	9.27 0.53 / 0.507	(x0.0767) (x1.79|0.497)	(x0.0583) (x0.982|0.266)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			9.48 0.115 / 0.282	(x0.0376) (x3.01|0.448)	(x0.0302) (x1.06|0.185)	9.39 0.139 / 0.358	(x0.0435) (x4.31|0.878)	(x0.0387) (x1.28|0.295)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			9.64 0.551 / 0.369	(x0.0495) (x1.97|0.534)	(x0.0364) (x0.961|0.209)	9.35 2.16 / 0.676	(x0.0806) (x1.06|0.459)	(x0.0449) (x0.839|0.255)
6	$gg \to t\bar{t} + (n-2)g$	N/A			9.32 0.349 / 0.429	(x0.0634) (x3.12|0.83)	(x0.0343) (x1.14|0.312)	17.8 1.93 / 1.66	(x0.127) (x1.34|0.619)	(x0.0398) (x0.787|0.39)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			9.29 0.144 / 0.299	(x0.0579) (x4.92|0.844)	(x0.0395) (x1.41|0.34)	13.2 0.629 / 1.01	(x0.105) (x2.48|0.902)	(x0.0523) (x0.919|0.359)
8	$gg \to gg + (n-2)g$	N/A			11 0.456 / 0.645	(x0.0701) (x1.94|0.596)	(x0.0343) (x0.947|0.322)	13.4 2.37 / 3.87	(x0.185) (x0.999|0.545)	(x0.0705) (x0.527|0.306)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			18.4 4.62 / 0.885	(x0.0764) (x0.855|0.457)	(x0.0292) (x0.763|0.261)
10	$d\bar{d} \to e^+e^- ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

gen O3 one-flow, hel. sum	7.64| x0.0417|	7.68| x0.0489|	7.66| x0.0453|	7.66 x0.0453|	 x0.0453	7.49| x0.0376|	11| x0.0701|	9.19| x0.0563|	9.4 x0.0593|	 x0.0566	7.44| x0.0435|	18.4| x0.185|	11.7| x0.101|	9.39 x0.0942|	 x0.104
gen O3 all-flows, one-hel	7.64| x0.0359|	7.68| x0.0401|	7.66| x0.038|	7.66 x0.038|	 x0.038	7.49| x0.0302|	11| x0.0481|	9.19| x0.039|	9.4 x0.0379|	 x0.0386	7.44| x0.0292|	18.4| x0.0705|	11.7| x0.0499|	9.39 x0.0509|	 x0.0479
run O3 one-flow, hel. sum	0.0319| x3.6|	0.126| x10.9|	0.079| x7.25|	0.079 x7.25|	 x5.07	0.115| x1.94|	0.551| x4.92|	0.33| x2.68|	0.327 x2.23|	 x2.39	0.139| x0.855|	4.62| x4.31|	1.61| x1.76|	1.28 x1.43|	 x1.2
run O3 all-flows, one-hel	0.202| x1.03|	0.203| x1.05|	0.202| x1.04|	0.202 x1.04|	 x1.04	0.282| x0.947|	0.645| x1.41|	0.375| x1.1|	0.347 x1.04|	 x1.08	0.358| x0.527|	3.87| x1.28|	1.19| x0.846|	0.885 x0.787|	 x0.728

UFO-SM LC process matrix (continued)

ID	base process	n=4			n=5			n=6		
1	$d\bar{d} \to Z + (n-1)g$	7.66 4.19 / 2.99	(x0.295) (x0.979|0.522)	(x0.165) (x0.595|0.279)	7.98 12.3 / 15.4	(x1.35) (x2.78|1.61)	(x2.06) (x0.458|0.268)	8.81 34.2 / 109	(x10.7) (x0.741|0.492)	(x7.74) (x0.727|0.351)
2	$u\bar{d} \to W^+ + (n-1)g$	7.51 2.52 / 3.07	(x0.214) (x1.15|0.571)	(x0.116) (x0.556|0.26)	7.81 8.33 / 15.7	(x6.79) (x1.34|0.862)	(x0.543) (x0.42|0.26)	8.1 23.8 / 96.8	(x7.63) (x0.937|0.592)	(x5.36) (x0.584|0.325)
3	$d\bar{d} \to e^+e^- + (n-2)g$	9.31 1.32 / 0.977	(x0.141) (x1.38|0.548)	(x0.0978) (x0.904|0.323)	9.92 3.82 / 3.52	(x0.323) (x3.55|1.98)	(x0.208) (x0.897|0.498)	10.2 11.2 / 16.9	(x1.87) (x0.8|0.495)	(x1.06) (x0.471|0.275)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.28 0.494 / 0.855	(x0.0751) (x2.11|0.648)	(x0.0435) (x0.923|0.309)	9.58 1.75 / 3.11	(x0.156) (x4.86|1.85)	(x0.0913) (x0.909|0.461)	9.81 6.05 / 15.2	(x0.575) (x1.22|0.847)	(x0.391) (x0.471|0.283)
5	$d\bar{d} \to ZZ + (n-2)g$	10.2 6.35 / 1.74	(x0.195) (x0.833|0.452)	(x0.074) (x0.623|0.244)	11.1 16.6 / 6.37	(x0.786) (x1.1|0.596)	(x0.242) (x0.474|0.231)	12.8 43.2 / 30.8	(x2.99) (x0.886|0.525)	(x1.54) (x0.373|0.217)
6	$gg \to t\bar{t} + (n-2)g$	32.4 6.89 / 8.56	(x0.366) (x0.968|0.562)	(x0.0972) (x0.643|0.428)	63.2 20.7 / 55.5	(x1.43) (x0.955|0.572)	(x0.513) (x0.957|0.54)	180 58.1 / 485	(x1.29) (x2.39|1.25)	(x2.15) (x1.31|0.562)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	17.8 2.44 / 4.19	(x0.36) (x1.79|0.997)	(x0.108) (x0.688|0.369)	24.1 8.13 / 23.7	(x1.74) (x1.8|1.08)	(x0.711) (x0.509|0.327)	33.2 23.7 / 157	(x2.44) (x1.69|1.14)	(x3.16) (x0.838|0.397)
8	$gg \to gg + (n-2)g$	16.5 9.26 / 26.9	(x0.883) (x0.705|0.469)	(x0.42) (x0.45|0.312)	18.9 28.5 / 219	(x1.59) (x0.836|0.477)	(x4.34) (x0.813|0.47)	26.7 79.5 / 3.86e3	(x4.18) (x0.872|0.594)	(x43.5) (x0.537|0.284)
9	$d\bar{d} \to ZZZ + (n-3)g$	19.8 12.9 / 1.58	(x0.15) (x0.683|0.425)	(x0.037) (x0.669|0.26)	26.6 33.4 / 3.92	(x0.71) (x0.797|0.491)	(x0.068) (x0.812|0.342)	37.4 83.8 / 14.5	(x0.865) (x1.25|0.666)	(x0.276) (x0.401|0.201)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	56.4 2.33 / 1.34	(x0.0303) (x1.27|0.731)	(x0.0167) (x0.766|0.303)	62 4.66 / 1.95	(x0.0409) (x1.67|1.19)	(x0.0199) (x0.864|0.352)	64.8 10.7 / 3.87	(x0.0792) (x0.886|0.587)	(x0.0319) (x0.635|0.29)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	31.9 3.06 / 1.75	(x0.0722) (x1.13|0.633)	(x0.0267) (x0.868|0.409)	63.4 9.04 / 4.71	(x0.204) (x1.23|0.841)	(x0.0403) (x1.07|0.554)	106 26.2 / 21.2	(x0.575) (x1.74|1.08)	(x0.0855) (x0.546|0.343)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	57.7 3.75 / 1.79	(x0.0561) (x0.801|0.459)	(x0.0385) (x0.624|0.275)	61.1 7.73 / 2.7	(x0.0665) (x0.641|0.401)	(x0.0885) (x0.805|0.37)	66 17.1 / 4.92	(x0.133) (x0.53|0.349)	(x0.0727) (x0.547|0.268)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	11.7 0.506 / 2.07	(x0.202) (x2.43|0.827)	(x0.137) (x0.797|0.358)	19.4 1.21 / 9.17	(x1.18) (x1.83|0.697)	(x0.426) (x0.627|0.347)	31.8 2.97 / 50.9	(x3.05) (x1.67|0.659)	(x1.24) (x1.08|0.514)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

gen O3 one-flow, hel. sum	7.51|	57.7|	22.2|	16.5		7.81|	63.4|	29.6|	19.4		8.1|	180|	45.9|	31.8	
	x0.0303|	x0.883|	x0.234|	x0.195|	x0.185	x0.0409|	x6.79|	x1.26|	x0.786|	x0.781	x0.0792|	x10.7|	x2.8|	x1.87|	x1.42
gen O3 all-flows, one-hel	7.51|	57.7|	22.2|	16.5		7.81|	63.4|	29.6|	19.4		8.1|	180|	45.9|	31.8	
	x0.0167|	x0.42|	x0.106|	x0.0972|	x0.0783	x0.0199|	x4.34|	x0.719|	x0.242|	x0.46	x0.0319|	x43.5|	x5.12|	x1.24|	x3.13
run O3 one-flow, hel. sum	0.494|	12.9|	4.31|	3.06		1.21|	33.4|	12|	8.33		2.97|	83.8|	32.4|	23.8	
	x0.683|	x2.43|	x1.25|	x1.13|	x0.932	x0.641|	x4.86|	x1.8|	x1.34|	x1.26	x0.53|	x2.39|	x1.2|	x0.937|	x1.25
run O3 all-flows, one-hel	0.855|	26.9|	4.45|	1.79		1.95|	219|	28|	6.37		3.87|	3.86e3|	374|	30.8	
	x0.45|	x0.923|	x0.701|	x0.669|	x0.573	x0.42|	x1.07|	x0.74|	x0.812|	x0.778	x0.373|	x1.31|	x0.655|	x0.547|	x0.632

UFO-SM LC process matrix (continued)

ID	base process	n=7				n=8				n=9			
1	$d\bar{d} \to Z + (n-1)g$	10.6 91.1 , 1.02e3	(x2.47) (x1.01|0.63)	(x78.3) (x0.624|0.262)		16.3 228 , 2.34e4	(N/A) N/A	(N/A) N/A		38.4 565 , 2.94e5	(N/A) N/A	(N/A) N/A	
2	$u\bar{d} \to W^+ + (n-1)g$	9.04 65.4 / 866	(x2.22) (x0.975|0.644)	(x57.5) (x0.716|0.305)		11.2 172 / 1.93e4	(N/A) N/A	(N/A) N/A		17.8 428 / 3.24e5	(N/A) N/A	(N/A) N/A	
3	$d\bar{d} \to e^+e^- + (n-2)g$	11.3 31.5 , 105	(x9.73) (x0.944|0.604)	(x10.3) (x0.723|0.348)		13.5 83.3 , 883	(x2.32) (x0.882|0.6)	(x70.6) (x0.971|0.376)		19.3 208 , 2.04e4	(N/A) N/A	(N/A) N/A	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	10.4 19.1 / 98.1	(x4.21) (x0.706|0.483)	(x3.44) (x0.592|0.316)		10.9 54 / 1.14e3	(x1.63) (x0.828|0.576)	(x34.1) (x0.64|0.228)		11.2 160 / 2.4e4	(N/A) N/A	(N/A) N/A	
5	$d\bar{d} \to ZZ + (n-2)g$	19.9 110 , 194	(x15) (x1.03|0.615)	(x9.91) (x0.657|0.264)		43.7 267 , 2.97e3	(N/A) N/A	(N/A) N/A		110 664 , 3.91e4	(N/A) N/A	(N/A) N/A	
6	$gg \to t\bar{t} + (n-2)g$	971 165 / 8.92e3	(x0.847) (x1.12|0.626)	(x5.13) (x0.592|0.259)		1.36e4 387 / 1.65e5	(N/A) N/A	(N/A) N/A		N/A			
7	$d\bar{d} \to t\bar{t} + (n-2)g$	64 65.4 , 2.04e3	(x4.87) (x2.12|1.34)	(x18.6) (x0.647|0.256)		317 171 , 2.5e4	(N/A) N/A	(N/A) N/A		N/A			
8	$gg \to gg + (n-2)g$	57.4 211 / 6.06e4	(N/A) N/A	(N/A) N/A		N/A				N/A			
9	$d\bar{d} \to ZZZ + (n-3)g$	84.1 204 , 67.2	(x1.87) (x1.38|0.814)	(x0.884) (x0.537|0.271)		254 473 , 425	(N/A) N/A	(N/A) N/A		745 1.33e3 , 8.14e3	(N/A) N/A	(N/A) N/A	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	82.1 25.9 / 12.6	(x0.203) (x0.775|0.537)	(x0.0782) (x0.467|0.244)		103 61.7 / 55.3	(x0.621) (x1.18|0.775)	(x0.408) (x0.553|0.292)		159 163 / 326	(x2.82) (x0.953|0.595)	(x2.11) (x0.58|0.247)	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	235 71.4 , 109	(x1.27) (x1.84|1.16)	(x0.287) (x0.926|0.429)		802 188 , 3.53e3	(x0.871) (x1.9|1.19)	(x0.746) (x0.208|0.0751)		7.32e3 466 , 1.43e4	(N/A) N/A	(N/A) N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	79.9 39.2 / 14.4	(x0.322) (x0.494|0.339)	(x0.18) (x0.439|0.237)		472 517 / 341	(x0.22) (x0.135|0.0817)	(x0.188) (x0.109|0.0541)		178 215 / 371	(x3.81) (x0.632|0.41)	(x3.17) (x0.57|0.251)	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	57.3 7.95 , 362	(x6) (x1.59|0.606)	(x5.44) (x0.978|0.428)		769 239 , 4.11e3	(N/A) N/A	(N/A) N/A		N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				UNSUPPORTED UNSUPPORTED	(N/A) N/A	(N/A) N/A		N/A			

gen O3 one-flow, hel. sum	9.04| x0.203|	971| x15|	130| x4.08|	57.4 x2.34|	x1.51	10.9| x0.22|	1.36e4| x2.32|	1.37e3| x1.13|	178 x0.871|	x0.653	11.2| x2.82|	7.32e3| x3.81|	956| x3.31|	110 x3.31|	x3.34
gen O3 all-flows, one-hel	9.04| x0.0782|	971| x78.3|	136| x15.8|	60.7 x5.29|	x5.11	10.9| x0.188|	802| x70.6|	280| x21.2|	103 x0.746|	x1.47	159| x2.11|	178| x3.17|	169| x2.64|	169 x2.64|	x2.67
run O3 one-flow, hel. sum	7.95| x0.494|	211| x2.12|	85.2| x1.16|	65.4 x1.02|	x1.23	54| x0.135|	517| x1.9|	237| x0.985|	208 x0.882|	x0.683	160| x0.632|	1.33e3| x0.953|	466| x0.793|	428 x0.793|	x0.771
run O3 all-flows, one-hel	12.6| x0.439|	6.06e4| x0.978|	5.73e3| x0.658|	194 x0.636|	x0.624	55.3| x0.109|	1.65e5| x0.971|	2.05e4| x0.496|	3.25e3 x0.553|	x0.401	326| x0.57|	3.24e5| x0.58|	8.05e4| x0.575|	2.04e4 x0.575|	x0.575

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.5 UFO-SM NLC process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	7.4 s 0.313 us	(x0.0581) (x1.32|0.185)	7.46 s 0.747 us	(x0.0659) (x1.13|0.303)	7.62 s 3.53 us	(x0.109) (x0.696|0.333)
2	$u\bar{d} \to W^+ + (n-1)g$	7.33 s 0.211 us	(x0.0524) (x1.79|0.186)	7.34 s 0.452 us	(x0.0672) (x1.59|0.322)	7.47 s 2.04 us	(x0.0801) (x0.873|0.371)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		9.18 s 0.481 us	(x0.0644) (x1.11|0.229)	9.71 s 1.21 us	(x0.0707) (x0.754|0.227)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		8.97 s 0.327 us	(x0.0462) (x1.37|0.154)	9.35 s 0.584 us	(x0.0518) (x1.2|0.218)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		8.56 s 0.936 us	(x0.0671) (x1.13|0.322)	9.33 s 2.9 us	(x0.0759) (x0.731|0.345)
6	$gg \to t\bar{t} + (n-2)g$	N/A		8.81 s 1.75 us	(x0.0565) (x0.791|0.285)	17.4 s 16.3 us	(x0.0957) (x0.554|0.298)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		9.2 s 1.17 us	(x0.0604) (x1.7|0.216)	13.1 s 6.29 us	(x0.0944) (x1.14|0.329)
8	$gg \to gg + (n-2)g$	N/A		11.2 s 7.96 us	(x0.0664) (x0.287|0.151)	13.3 s 120 us	(x0.478) (x0.385|0.179)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		17.8 s 5.67 us	(x0.0575) (x0.629|0.369)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		7.33| (x0.0524)	7.4| (x0.0581) |(x0.0553)|(x0.0553)|(x0.0553)	7.34| (x0.0462)	11.2| (x0.0672) |(x0.0618)|(x0.0651)|(x0.0617)	7.47| (x0.0518)	17.8| (x0.478) |(x0.124)|(x0.0801)| (x0.129)
		7.36| (x0.0553)	7.36 (x0.0553)	8.84| (x0.0618)	8.89 (x0.0651)|(x0.0617)	11.7| (x0.124)	9.71 (x0.0801)| (x0.129)
summary: run wall		0.211| (x1.32)	0.313| (x1.79) |(x1.55)|(x1.55)| (x1.51)	0.327| (x0.287)	7.96| (x1.7) |(x1.14)|(x1.13)| (x0.67)	0.584| (x0.385)	120| (x1.2) |(x0.773)|(x0.731)|(x0.466)
		0.262| (x1.55)	0.262 (x1.55)| (x1.51)	1.73| (x1.14)	0.841 (x1.13)| (x0.67)	17.6| (x0.773)	3.53 (x0.731)|(x0.466)
summary: run eval		0.211| (x0.185)	0.313| (x0.186) |(x0.186)|(x0.186)| (x0.185)	0.327| (x0.151)	7.96| (x0.322) |(x0.248)|(x0.257)| (x0.202)	0.584| (x0.179)	120| (x0.371) |(x0.297)|(x0.329)|(x0.214)
		0.262| (x0.262)	0.262 (x0.186)| (x0.185)	1.73| (x1.73)	0.841 (x0.841)| (x0.202)	17.6| (x0.297)	3.53 (x0.329)|(x0.214)

UFO-SM NLC process matrix (continued)

ID	base process	n=4		n=5	
1	$d\bar{d} \to Z + (n-1)g$	8 s 31.3 us	(x0.404) (x0.672|0.348)	7.88 s 365 us	(x3.42) (x0.836|0.335)
2	$u\bar{d} \to W^+ + (n-1)g$	7.4 s 18.4 us	(x0.275) (x0.574|0.338)	7.71 s 233 us	(x2.34) (x0.826|0.343)
3	$d\bar{d} \to e^+e^- + (n-2)g$	10 s 5.45 us	(x0.104) (x0.428|0.207)	10.1 s 44.7 us	(x0.191) (x0.307|0.188)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.4 s 2.53 us	(x0.0701) (x0.604|0.242)	9.76 s 22.4 us	(x0.112) (x0.299|0.191)
5	$d\bar{d} \to ZZ + (n-2)g$	11.3 s 15.3 us	(x0.155) (x0.516|0.32)	10.8 s 117 us	(x0.844) (x0.779|0.419)
6	$gg \to t\bar{t} + (n-2)g$	31.4 s 217 us	(x0.61) (x0.78|0.33)	63.3 s 3.44e3 us	(x3.85) (x0.87|0.259)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	18.7 s 56.9 us	(x0.428) (x1.32|0.422)	28.1 s 694 us	(x2.41) (x1.18|0.41)
8	$gg \to gg + (n-2)g$	16.6 s 2.02e3 us	(x4.78) (x0.412|0.177)	22.8 s 3.69e4 us	(x60) (x0.476|0.155)
9	$d\bar{d} \to ZZZ + (n-3)g$	19.9 s 15.3 us	(x0.0702) (x0.514|0.364)	25.2 s 75.8 us	(x0.242) (x0.716|0.425)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	58.4 s 3.94 us	(x0.0108) (x0.657|0.424)	63.7 s 8.81 us	(x0.0135) (x0.531|0.363)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	32.8 s 11.7 us	(x0.0373) (x0.616|0.373)	68 s 64.9 us	(x0.1) (x0.889|0.419)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	59.6 s 7.48 us	(x0.0099) (x0.33|0.217)	65.7 s 15.6 us	(x0.0126) (x0.287|0.2)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	12.2 s 141 us	(x0.0623) (x0.0359|0.0149)	23.8 s 1.3e3 us	(x0.25) (x0.0502|0.017)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A	

summary: gen	7.4| (x0.0099)|	59.6| (x4.78)|	22.7| (x0.54)|	16.6 (x0.104)| (x0.405)	7.71| (x0.0126)|	68| (x60)|	31.3| (x5.67)|	23.8 (x0.25)| (x4.32)
summary: run wall	2.53| (x0.0359)|	2.02e3| (x1.32)|	196| (x0.574)|	15.3 (x0.574)| (x0.45)	8.81| (x0.0502)|	3.69e4| (x1.18)|	3.33e3| (x0.619)|	117 (x0.716)| (x0.512)
summary: run eval	2.53| (x0.0149)|	2.02e3| (x0.424)|	196| (x0.29)|	15.3 (x0.33)| (x0.193)	8.81| (x0.017)|	3.69e4| (x0.425)|	3.33e3| (x0.287)|	117 (x0.335)| (x0.167)

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.6 UFO-SM full-colour process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	7.29 s 0.347 us	(x0.061) (x1.2|0.17)	7.11 s 0.817 us	(x0.0692) (x1.07|0.282)	7.45 s 3.99 us	(x0.0994) (x0.613|0.294)
2	$u\bar{d} \to W^+ + (n-1)g$	7.33 s 0.159 us	(x0.0523) (x2.39|0.247)	7.1 s 0.496 us	(x0.0621) (x1.49|0.306)	7.53 s 2.17 us	(x0.081) (x0.825|0.346)
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		9.34 s 0.616 us	(x0.0633) (x0.871|0.18)	9.32 s 1.28 us	(x0.0746) (x0.692|0.201)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		9.46 s 0.354 us	(x0.0512) (x1.27|0.143)	9.07 s 0.65 us	(x0.0516) (x1.07|0.193)
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		8.93 s 1.07 us	(x0.0558) (x0.952|0.286)	9.55 s 3.1 us	(x0.0715) (x0.686|0.325)
6	$gg \to t\bar{t} + (n-2)g$	N/A		8.44 s 1.96 us	(x0.062) (x0.709|0.255)	16.9 s 19.6 us	(x0.103) (x0.512|0.254)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		8.7 s 1.53 us	(x0.0774) (x1.36|0.172)	13.1 s 7.39 us	(x0.0953) (x0.972|0.28)
8	$gg \to gg + (n-2)g$	N/A		9.91 s 10 us	(x0.0818) (x0.24|0.125)	13.2 s 148 us	(x0.49) (x0.359|0.146)
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		17.5 s 5.98 us	(x0.0616) (x0.599|0.35)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		7.29| (x0.0523)	7.33| (x0.061) |(x0.0567) |(x0.0567) |(x0.0567)	7.1| (x0.0512)	9.91| (x0.0818) |(x0.0653) |(x0.0627) |(x0.0654)	7.45| (x0.0516)	17.5| (x0.49) |(x0.125) |(x0.081) |(x0.132)
summary: run wall		0.159| (x1.2)	0.347| (x2.39) |0.253| (x1.8) |0.253 (x1.8) |(x1.58)	0.354| (x0.24)	10| (x1.49) |2.11| (x0.995) |0.945 (x1.01) |(x0.563)	0.65| (x0.359)	148| (x1.07) |21.4| (x0.703) |3.99 (x0.686) |(x0.426)
summary: run eval		0.159| (x0.17)	0.347| (x0.247) |0.253| (x0.208) |0.253 (x0.208) |(x0.194)	0.354| (x0.125)	10| (x0.306) |2.11| (x0.219) |0.945 (x0.217) | (x0.17)	0.65| (x0.146)	148| (x0.35) |21.4| (x0.265) |3.99 (x0.28) |(x0.178)

UFO-SM full-colour process matrix (continued)

ID	base process	n=4		n=5	
1	$d\bar{d} \to Z + (n-1)g$	7.68 s 35.9 us	(x0.292) (x0.596|0.3)	7.97 s 465 us	(x3.45) (x0.805|0.273)
2	$u\bar{d} \to W^+ + (n-1)g$	7.52 s 20.6 us	(x0.194) (x0.511|0.296)	7.67 s 281 us	(x2.38) (x0.801|0.291)
3	$d\bar{d} \to e^+e^- + (n-2)g$	9.57 s 6.05 us	(x0.0479) (x0.334|0.191)	10.1 s 50.7 us	(x0.203) (x0.357|0.192)
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	9.35 s 3.08 us	(x0.0403) (x0.366|0.174)	9.64 s 25.4 us	(x0.111) (x0.271|0.168)
5	$d\bar{d} \to ZZ + (n-2)g$	9.86 s 17.1 us	(x0.116) (x0.441|0.285)	10.8 s 131 us	(x0.864) (x0.721|0.373)
6	$gg \to t\bar{t} + (n-2)g$	31.9 s 286 us	(x0.519) (x0.73|0.246)	64.2 s 6.03e3 us	(x4.08) (x1.04|0.152)
7	$d\bar{d} \to t\bar{t} + (n-2)g$	18.8 s 74.1 us	(x0.286) (x0.924|0.32)	28.3 s 1.09e3 us	(x2.41) (x0.883|0.26)
8	$gg \to gg + (n-2)g$	16.3 s 3.08e3 us	(x5.45) (x0.457|0.117)	22.5 s 9.82e4 us	(x90.7) (x0.811|0.0698)
9	$d\bar{d} \to ZZZ + (n-3)g$	19.9 s 16.1 us	(x0.0703) (x0.49|0.35)	25.8 s 78.8 us	(x0.233) (x0.695|0.398)
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	58.4 s 4.06 us	(x0.0109) (x0.641|0.419)	63.4 s 9.11 us	(x0.0135) (x0.505|0.347)
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	33.2 s 12.6 us	(x0.0375) (x0.577|0.375)	67.4 s 68.6 us	(x0.101) (x0.838|0.392)
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	60 s 7.5 us	(x0.0101) (x0.329|0.219)	65.6 s 15.9 us	(x0.0125) (x0.282|0.196)
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	12.4 s 148 us	(x0.0611) (x0.0344|0.0141)	23.8 s 1.31e3 us	(x0.247) (x0.0496|0.0168)
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A	
	summary: gen	7.52| 60| 22.7| 16.3 (x0.0101)| (x5.45)| (x0.549)|(x0.0703)| (x0.41)		7.67| 67.4| 31.3| 23.8 (x0.0125)| (x90.7)| (x8.06)| (x0.247)| (x6.02)	
	summary: run wall	3.08| 3.08e3| 286| 17.1 (x0.0344)| (x0.924)| (x0.495)| (x0.49)| (x0.472)		9.11| 9.82e4| 8.29e3| 131 (x0.0496)| (x1.04)| (x0.62)| (x0.721)| (x0.814)	
	summary: run eval	3.08| 3.08e3| 286| 17.1 (x0.0141)| (x0.419)| (x0.254)| (x0.285)| (x0.133)		9.11| 9.82e4| 8.29e3| 131 (x0.0168)| (x0.398)| (x0.241)| (x0.26)|(x0.0781)	

Cell ordering and units follow the reading guide at the start of this section. The first/second LC slots use the complete-coverage topology-replay selected-flow/helicity-sum and all-flow-union all-flows/fixed-helicity layouts, respectively. A vertical bar separates the pyAmpliCol Rusticol-core wall and evaluator-only multipliers. Every multiplier is pyAmpliCol divided by AmpliCol.

5.7 UFO-SM eager-DAG JIT O3 versus compiled JIT O3 LC process matrix

ID	base process	n=1			n=2			n=3		
1	$d\bar{d} \to Z + (n-1)g$	0.376 , 0.308 0.454 , 0.209	(x1.46) (x1.09|0.367)	(x1.67) (x1.84|0.256)	0.454 , 0.36 0.904 , 0.371	(N/A) N/A	(N/A) N/A	0.897 , 0.481 1.82 , 0.67	(N/A) N/A	(N/A) N/A
2	$u\bar{d} \to W^+ + (n-1)g$	0.319 / 0.274 0.348 / 0.214	(N/A) N/A	(N/A) N/A	0.371 / 0.312 0.705 / 0.37	(N/A) N/A	(N/A) N/A	0.701 / 0.378 1.26 / 0.654	(N/A) N/A	(N/A) N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A			0.601 , 0.461 0.572 , 0.326	(N/A) N/A	(N/A) N/A	0.711 , 0.54 0.951 , 0.498	(N/A) N/A	(N/A) N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A			0.356 / 0.286 0.347 / 0.299	(N/A) N/A	(N/A) N/A	0.408 / 0.363 0.597 / 0.458	(N/A) N/A	(N/A) N/A
5	$d\bar{d} \to ZZ + (n-2)g$	N/A			0.477 , 0.351 1.08 , 0.355	(N/A) N/A	(N/A) N/A	0.754 , 0.42 2.3 , 0.568	(N/A) N/A	(N/A) N/A
6	$gg \to t\bar{t} + (n-2)g$	N/A			0.591 / 0.32 1.09 / 0.488	(N/A) N/A	(N/A) N/A	2.26 / 0.707 2.59 / 1.31	(N/A) N/A	(N/A) N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A			0.538 , 0.367 0.707 , 0.42	(N/A) N/A	(N/A) N/A	1.39 , 0.691 1.56 , 0.93	(N/A) N/A	(N/A) N/A
8	$gg \to gg + (n-2)g$	N/A			0.774 / 0.379 0.882 / 0.61	(N/A) N/A	(N/A) N/A	2.48 / 0.944 2.37 / 2.04	(N/A) N/A	(N/A) N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A			N/A			1.41 , 0.538 3.95 , 0.675	(N/A) N/A	(N/A) N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A			N/A			N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A			N/A			N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A			N/A			N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A			N/A			N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		

gen O3 one-flow, hel. sum	0.319| x1.46|	0.376| x1.46|	0.347| x1.46|	0.347 x1.46|	x1.46	0.356| N/A	0.774|	0.52|	0.508	0.408| N/A	2.48|	1.22|	0.897
gen O3 all-flows, one-hel	0.308| x1.67|	0.308| x1.67|	0.308| x1.67|	0.308 x1.67|	x1.67	N/A N/A				N/A N/A			
run O3 one-flow, hel. sum	0.348| x1.09|	0.454| x1.09|	0.401| x1.09|	0.401 x1.09|	x1.09	0.347| N/A	1.09|	0.786|	0.794	0.597| N/A	3.95|	1.93|	1.82
run O3 all-flows, one-hel	0.209| x1.84|	0.214| x1.84|	0.211| x1.84|	0.211 x1.84|	x1.84	0.299| N/A	0.61|	0.405|	0.37	0.458| N/A	2.04|	0.866|	0.67

UFO-SM eager-DAG JIT O3 versus compiled JIT O3 LC process matrix (continued)

ID	base process	n=4			n=5			n=6		
1	$d\bar{d} \to Z + (n-1)g$	2.26 , 1.26 4.1 , 1.78	(N/A) N/A	(N/A) N/A	10.8 , 16.4 34.2 , 7.03	(N/A) N/A	(N/A) N/A	94 , 68.2 25.3 , 79.4	(N/A) N/A	(N/A) N/A
2	$u\bar{d} \to W^+ + (n-1)g$	1.61 / 0.872 2.9 / 1.71	(N/A) N/A	(N/A) N/A	53.1 / 4.24 11.1 / 6.58	(N/A) N/A	(N/A) N/A	61.8 / 43.4 22.4 / 56.5	(N/A) N/A	(N/A) N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	1.32 , 0.91 1.83 , 0.883	(N/A) N/A	(N/A) N/A	3.2 , 2.07 13.6 , 3.16	(N/A) N/A	(N/A) N/A	19 , 10.8 8.95 , 7.96	(N/A) N/A	(N/A) N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	0.697 / 0.404 1.04 / 0.789	(N/A) N/A	(N/A) N/A	1.49 / 0.875 8.49 / 2.83	(N/A) N/A	(N/A) N/A	5.64 / 3.84 7.37 / 7.16	(N/A) N/A	(N/A) N/A
5	$d\bar{d} \to ZZ + (n-2)g$	1.98 , 0.752 5.29 , 1.09	(N/A) N/A	(N/A) N/A	8.73 , 2.69 18.3 , 3.02	(N/A) N/A	(N/A) N/A	38.4 , 19.7 38.2 , 11.5	(N/A) N/A	(N/A) N/A
6	$gg \to t\bar{t} + (n-2)g$	11.9 / 3.15 6.67 / 5.51	(N/A) N/A	(N/A) N/A	90.5 / 32.4 19.8 / 53.1	(N/A) N/A	(N/A) N/A	233 / 387 139 / 635	(N/A) N/A	(N/A) N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	6.41 , 1.92 4.37 , 2.88	(N/A) N/A	(N/A) N/A	41.9 , 17.1 14.6 , 12.1	(N/A) N/A	(N/A) N/A	81.1 , 105 40.1 , 132	(N/A) N/A	(N/A) N/A
8	$gg \to gg + (n-2)g$	14.6 / 6.92 6.53 / 12.1	(N/A) N/A	(N/A) N/A	30.2 / 82.3 23.9 / 178	(N/A) N/A	(N/A) N/A	112 / 1.16e3 69.4 / 2.07e3	(N/A) N/A	(N/A) N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	2.98 , 0.733 8.79 , 1.06	(N/A) N/A	(N/A) N/A	18.9 , 1.81 26.6 , 3.18	(N/A) N/A	(N/A) N/A	32.3 , 10.3 104 , 5.83	(N/A) N/A	(N/A) N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	1.71 / 0.945 2.97 / 1.03	(N/A) N/A	(N/A) N/A	2.54 / 1.23 7.8 / 1.68	(N/A) N/A	(N/A) N/A	5.13 / 2.07 9.45 / 2.46	(N/A) N/A	(N/A) N/A
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	2.3 , 0.849 3.45 , 1.52	(N/A) N/A	(N/A) N/A	12.9 , 2.55 11.1 , 5.05	(N/A) N/A	(N/A) N/A	61.2 , 9.1 45.4 , 11.6	(N/A) N/A	(N/A) N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	3.24 / 2.22 3 / 1.12	(N/A) N/A	(N/A) N/A	4.07 / 5.41 4.95 / 2.17	(N/A) N/A	(N/A) N/A	8.75 / 4.8 9.06 / 2.69	(N/A) N/A	(N/A) N/A
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	2.37 , 1.61 1.23 , 1.65	(N/A) N/A	(N/A) N/A	22.8 , 8.26 2.22 , 5.75	(N/A) N/A	(N/A) N/A	97 , 39.3 4.96 , 54.9	(N/A) N/A	(N/A) N/A
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A			N/A			N/A		
gen O3 one-flow, hel. sum		0.697| 14.6| 4.1| 2.3 N/A			1.49| 90.5| 23.2| 12.9 N/A			5.13| 233| 65.3| 61.2 N/A		
gen O3 all-flows, one-hel		N/A N/A			N/A N/A			N/A N/A		
run O3 one-flow, hel. sum		1.04| 8.79| 4.01| 3.45 N/A			2.22| 34.2| 15.1| 13.6 N/A			4.96| 139| 40.3| 25.3 N/A		
run O3 all-flows, one-hel		0.789| 12.1| 2.55| 1.52 N/A			1.68| 178| 21.8| 5.05 N/A			2.46| 2.07e3| 237| 11.6 N/A		

UFO-SM eager-DAG JIT O3 versus compiled JIT O3 LC process matrix (continued)

ID	base process	n=7				n=8				n=9			
1	$d\bar{d} \to Z + (n-1)g$	26.1 / 827 92 / 635	(N/A) N/A	(N/A) N/A		N/A				60.4 / 6.79e3 491 / 6.92e4	(N/A) N/A	(N/A) N/A	
2	$u\bar{d} \to W^+ + (n-1)g$	20 / 520 63.8 / 620	(N/A) N/A	(N/A) N/A		N/A				51.3 / 6.73e3 407 / 7.72e4	(N/A) N/A	(N/A) N/A	
3	$d\bar{d} \to e^+e^- + (n-2)g$	110 / 117 29.7 / 75.9	(N/A) N/A	(N/A) N/A		31.4 / 956 73.4 / 858	(N/A) N/A	(N/A) N/A		22.3 / 554 208 / 6.1e3	(N/A) N/A	(N/A) N/A	
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	43.9 / 35.9 13.5 / 58.1	(N/A) N/A	(N/A) N/A		17.9 / 373 44.7 / 726	(N/A) N/A	(N/A) N/A		17.7 / 566 157 / 6.61e3	(N/A) N/A	(N/A) N/A	
5	$d\bar{d} \to ZZ + (n-2)g$	298 / 197 113 / 127	(N/A) N/A	(N/A) N/A		24.7 / 78.2 230 / 863	(N/A) N/A	(N/A) N/A		58.9 / 719 541 / 9.81e3	(N/A) N/A	(N/A) N/A	
6	$gg \to t\bar{t} + (n-2)g$	822 / 4.98e3 185 / 5.28e3	(N/A) N/A	(N/A) N/A		N/A				N/A			
7	$d\bar{d} \to t\bar{t} + (n-2)g$	311 / 1.19e3 139 / 1.32e3	(N/A) N/A	(N/A) N/A		N/A				N/A			
8	$gg \to gg + (n-2)g$	N/A				N/A				N/A			
9	$d\bar{d} \to ZZZ + (n-3)g$	157 / 74.4 281 / 36.1	(N/A) N/A	(N/A) N/A		43.6 / 17.2 399 / 165	(N/A) N/A	(N/A) N/A		104 / 131 1.18e3 / 1.63e3	(N/A) N/A	(N/A) N/A	
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	16.6 / 6.42 20.1 / 5.87	(N/A) N/A	(N/A) N/A		63.9 / 42 72.6 / 30.5	(N/A) N/A	(N/A) N/A		449 / 337 156 / 189	(N/A) N/A	(N/A) N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	299 / 67.4 131 / 101	(N/A) N/A	(N/A) N/A		698 / 599 357 / 733	(N/A) N/A	(N/A) N/A		87.9 / 480 613 / 4.31e3	(N/A) N/A	(N/A) N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	25.7 / 14.4 19.4 / 6.33	(N/A) N/A	(N/A) N/A		104 / 88.6 69.7 / 37.3	(N/A) N/A	(N/A) N/A		680 / 565 136 / 212	(N/A) N/A	(N/A) N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	344 / 312 12.6 / 354	(N/A) N/A	(N/A) N/A		3.84 / 252 15.2 / 2.25e3	(N/A) N/A	(N/A) N/A		N/A			
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				4.95 / 88 25.3 / 888	(N/A) N/A	(N/A) N/A		N/A			
gen O3 one-flow, hel. sum		16.6 | 822 | 206 | 134 N/A				3.84 | 698 | 110 | 31.4 N/A				17.7 | 680 | 170 | 60.4 N/A			
gen O3 all-flows, one-hel		N/A N/A				N/A N/A				N/A N/A			
run O3 one-flow, hel. sum		12.6 | 281 | 91.7 | 77.9 N/A				15.2 | 399 | 143 | 72.6 N/A				136 | 1.18e3 | 432 | 407 N/A			
run O3 all-flows, one-hel		5.87 | 5.28e3 | 718 | 114 N/A				30.5 | 2.25e3 | 728 | 733 N/A				189 | 7.72e4 | 1.95e4 | 6.1e3 N/A			

Cell ordering and units match the preceding process matrices, but compiled UFO-SM JIT O3 is the reference and eager-DAG JIT O3 is the candidate. The first/second LC slots compare topology-replay and all-flow-union artifacts with like-for-like layouts. A vertical bar separates the eager Rusticol-core wall and inclusive eager-execution multipliers. Every multiplier is eager divided by compiled.

5.8 UFO-SM eager-DAG JIT O3 versus compiled JIT O3 NLC process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	0.43 s 0.412 us	(x1.28) (x1.12|0.352)	0.491 s 0.843 us	N/A N/A	0.832 s 2.46 us	N/A N/A
2	$u\bar{d} \to W^+ + (n-1)g$	0.384 s 0.376 us	N/A N/A	0.493 s 0.719 us	N/A N/A	0.598 s 1.78 us	N/A N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		0.591 s 0.534 us	N/A N/A	0.687 s 0.914 us	N/A N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		0.415 s 0.449 us	N/A N/A	0.485 s 0.701 us	N/A N/A
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		0.575 s 1.05 us	N/A N/A	0.708 s 2.12 us	N/A N/A
6	$gg \to t\bar{t} + (n-2)g$	N/A		0.498 s 1.38 us	N/A N/A	1.67 s 9.03 us	N/A N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		0.556 s 1.99 us	N/A N/A	1.24 s 7.15 us	N/A N/A
8	$gg \to gg + (n-2)g$	N/A		0.741 s 2.29 us	N/A N/A	6.36 s 46.1 us	N/A N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		1.02 s 3.57 us	N/A N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		0.384| (x1.28)|	0.43| (x1.28)| 0.407| (x1.28)| 0.407 (x1.28)| (x1.28)	0.415| N/A	0.741| 0.545| 0.527	0.485| N/A	6.36| 1.51| 0.832
summary: run wall		0.376| (x1.12)|	0.412| (x1.12)| 0.394| (x1.12)| 0.394 (x1.12)| (x1.12)	0.449| N/A	2.29| 1.16| 0.949	0.701| N/A	46.1| 8.21| 2.46
summary: run eval		0.376| (x0.352)|	0.412| (x0.352)| 0.394| (x0.352)| 0.394 (x0.352)| (x0.352)	0.449| N/A	2.29| 1.16| 0.949	0.701| N/A	46.1| 8.21| 2.46

UFO-SM eager-DAG JIT O3 versus compiled JIT O3 NLC process matrix (continued)

ID	base process	n=4		n=5	
1	$d\bar{d} \to Z + (n-1)g$	3.24 s 21 us	N/A N/A	26.9 s 305 us	N/A N/A
2	$u\bar{d} \to W^+ + (n-1)g$	2.03 s 10.6 us	N/A N/A	18.1 s 192 us	N/A N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	1.04 s 2.33 us	N/A N/A	1.92 s 13.7 us	N/A N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	0.66 s 1.53 us	N/A N/A	1.09 s 6.69 us	N/A N/A
5	$d\bar{d} \to ZZ + (n-2)g$	1.76 s 7.91 us	N/A N/A	9.15 s 91.4 us	N/A N/A
6	$gg \to t\bar{t} + (n-2)g$	19.1 s 169 us	N/A N/A	244 s 2.99e3 us	N/A N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	8.02 s 75.1 us	N/A N/A	67.6 s 818 us	N/A N/A
8	$gg \to gg + (n-2)g$	79.2 s 834 us	N/A N/A	1.37e3 s 1.75e4 us	N/A N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	1.4 s 7.89 us	N/A N/A	6.08 s 54.3 us	N/A N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	0.632 s 2.59 us	N/A N/A	0.859 s 4.68 us	N/A N/A
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	1.22 s 7.19 us	N/A N/A	6.81 s 57.6 us	N/A N/A
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	0.59 s 2.47 us	N/A N/A	0.825 s 4.48 us	N/A N/A
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	0.762 s 5.05 us	N/A N/A	5.93 s 65.4 us	N/A N/A
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A	
summary: gen		0.59| 79.2| 9.21| 1.4 N/A		0.825| 1.37e3| 135| 6.81 N/A	
summary: run wall		1.53| 834| 88.2| 7.89 N/A		4.48| 1.75e4| 1.7e3| 65.4 N/A	
summary: run eval		1.53| 834| 88.2| 7.89 N/A		4.48| 1.75e4| 1.7e3| 65.4 N/A	

Cell ordering and units match the preceding process matrices, but compiled UFO-SM JIT O3 is the reference and eager-DAG JIT O3 is the candidate. The first/second LC slots compare topology-replay and all-flow-union artifacts with like-for-like layouts. A vertical bar separates the eager Rusticol-core wall and inclusive eager-execution multipliers. Every multiplier is eager divided by compiled.

5.9 UFO-SM eager-DAG JIT O3 versus compiled JIT O3 full-colour process matrix

ID	base process	n=1		n=2		n=3	
1	$d\bar{d} \to Z + (n-1)g$	0.445 s 0.418 us	(x1.24) (x1.1|0.345)	0.491 s 0.871 us	N/A N/A	0.74 s 2.44 us	N/A N/A
2	$u\bar{d} \to W^+ + (n-1)g$	0.384 s 0.38 us	N/A N/A	0.441 s 0.74 us	N/A N/A	0.61 s 1.79 us	N/A N/A
3	$d\bar{d} \to e^+e^- + (n-2)g$	N/A		0.591 s 0.536 us	N/A N/A	0.695 s 0.885 us	N/A N/A
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	N/A		0.485 s 0.448 us	N/A N/A	0.468 s 0.696 us	N/A N/A
5	$d\bar{d} \to ZZ + (n-2)g$	N/A		0.499 s 1.02 us	N/A N/A	0.683 s 2.12 us	N/A N/A
6	$gg \to t\bar{t} + (n-2)g$	N/A		0.523 s 1.39 us	N/A N/A	1.74 s 10 us	N/A N/A
7	$d\bar{d} \to t\bar{t} + (n-2)g$	N/A		0.673 s 2.09 us	N/A N/A	1.25 s 7.19 us	N/A N/A
8	$gg \to gg + (n-2)g$	N/A		0.81 s 2.42 us	N/A N/A	6.46 s 53.3 us	N/A N/A
9	$d\bar{d} \to ZZZ + (n-3)g$	N/A		N/A		1.08 s 3.58 us	N/A N/A
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	N/A		N/A		N/A	
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	N/A		N/A		N/A	
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	N/A		N/A		N/A	
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	N/A		N/A		N/A	
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A		N/A		N/A	
summary: gen		0.384| (x1.24)	0.445| (x1.24) | 0.414| (x1.24) | 0.414 (x1.24) | (x1.24)	0.441| N/A	0.81| 0.564| 0.511	0.468| N/A	6.46| 1.53| 0.74
summary: run wall		0.38| (x1.1)	0.418| (x1.1) | 0.399| (x1.1) | 0.399 (x1.1) | (x1.1)	0.448| N/A	2.42| 1.19| 0.947	0.696| N/A	53.3| 9.11| 2.44
summary: run eval		0.38| (x0.345)	0.418| (x0.345) | 0.399| (x0.345) | 0.399 (x0.345) | (x0.345)	0.448| N/A	2.42| 1.19| 0.947	0.696| N/A	53.3| 9.11| 2.44

UFO-SM eager-DAG JIT O3 versus compiled JIT O3 full-colour process matrix (continued)

ID	base process	n=4				n=5			
1	$d\bar{d} \to Z + (n-1)g$	2.24 s 21.4 us	N/A N/A			27.5 s 374 us	N/A N/A		
2	$u\bar{d} \to W^+ + (n-1)g$	1.46 s 10.5 us	N/A N/A			18.3 s 225 us	N/A N/A		
3	$d\bar{d} \to e^+e^- + (n-2)g$	0.458 s 2.02 us	N/A N/A			2.04 s 18.1 us	N/A N/A		
4	$u\bar{d} \to e^+\nu_e + (n-2)g$	0.377 s 1.13 us	N/A N/A			1.07 s 6.87 us	N/A N/A		
5	$d\bar{d} \to ZZ + (n-2)g$	1.15 s 7.55 us	N/A N/A			9.33 s 94.7 us	N/A N/A		
6	$gg \to t\bar{t} + (n-2)g$	16.6 s 209 us	N/A N/A			262 s 6.25e3 us	N/A N/A		
7	$d\bar{d} \to t\bar{t} + (n-2)g$	5.38 s 68.5 us	N/A N/A			68.3 s 962 us	N/A N/A		
8	$gg \to gg + (n-2)g$	88.6 s 1.41e3 us	N/A N/A			2.04e3 s 7.96e4 us	N/A N/A		
9	$d\bar{d} \to ZZZ + (n-3)g$	1.4 s 7.92 us	N/A N/A			6 s 54.8 us	N/A N/A		
10	$d\bar{d} \to e^+e^-ZH + (n-4)g$	0.634 s 2.6 us	N/A N/A			0.854 s 4.6 us	N/A N/A		
11	$d\bar{d} \to t\bar{t}ZH + (n-4)g$	1.25 s 7.26 us	N/A N/A			6.84 s 57.5 us	N/A N/A		
12	$d\bar{d} \to e^+e^-e^+e^- + (n-4)g$	0.608 s 2.47 us	N/A N/A			0.823 s 4.48 us	N/A N/A		
13	$d\bar{d} \to u\bar{u}\,s\bar{s} + (n-4)g$	0.76 s 5.07 us	N/A N/A			5.88 s 65.2 us	N/A N/A		
14	$d\bar{d} \to u\bar{u}\,s\bar{s}\,c\bar{c} + (n-6)g$	N/A				N/A			
summary: gen		0.377|	88.6|	9.3|	1.25 N/A	0.823|	2.04e3|	189|	6.84 N/A
summary: run wall		1.13|	1.41e3|	135|	7.55 N/A	4.48|	7.96e4|	6.75e3|	65.2 N/A
summary: run eval		1.13|	1.41e3|	135|	7.55 N/A	4.48|	7.96e4|	6.75e3|	65.2 N/A

Cell ordering and units match the preceding process matrices, but compiled UFO-SM JIT O3 is the reference and eager-DAG JIT O3 is the candidate. The first/second LC slots compare topology-replay and all-flow-union artifacts with like-for-like layouts. A vertical bar separates the eager Rusticol-core wall and inclusive eager-execution multipliers. Every multiplier is eager divided by compiled.

6 Z-plus-Jets Evaluator Ladders

The dedicated $d\bar{d} \to Z + (n-1)g$ ladders compare the independent reference with compiled JIT level 1, compiled JIT level 3, eager-DAG JIT O3, ASM O3, and C++ O3 evaluator configurations. They retain final-state multiplicities $n = 1, \ldots, 9$ for both the built-in model and the UFO-SM result set. Each row states its evaluator backend and optimization level explicitly; all other workload settings follow the benchmark contract above.

The eager row starts from the corresponding prepared model bundle and creates two complete process artifacts: topology replay for selected-flow/helicity-sum and all-flow union for all-flows/fixed-helicity. Each workload block reports its own process generation time and applies its flow or helicity selector at runtime. The displayed Z-ladder ratios continue to use original `AmpliCol` as the reference. In addition, each eager workload is checked pointwise against compiled JIT O3 at the same selector and layout with the stricter eager-comparison tolerance. Prepared-model creation is excluded from both process-generation entries and retained as separate provenance.

Runtime comparisons between these backends also include their vectorization strategy. The SymJIT application evaluates a batch with SIMD across phase-space points, whereas generated ASM and C++ payloads are scalar and Rusticol invokes them point by point. Their ratios therefore measure both symbolic/code-generation quality and this SIMD advantage; they must not be interpreted as a comparison of otherwise identical vectorized kernels.

Worked $d\bar{d} \to Zg$ example

For $n = 2$, the explicit process is

$$d(p_1)\,\bar{d}(p_2) \to Z(p_3)\,g(p_4).$$

The report labels source particles in process order. The selected leading-colour reference order for the selected-flow timing is therefore $(2, 4, 1, 3)$: the colour line enters through $\bar{d}$, traverses the gluon, and exits through d, while the colour-singlet Z remains in the public ordering only to keep source labels unambiguous. Internally, the colour-sector match is the coloured word $(2, 4, 1)$.

`pyAmpliCol` builds a shared-current DAG for the selected process. Source nodes carry particle, momentum, helicity, flavour, and colour-flow labels; composite nodes are keyed by the full current identity and the external-label mask. When two diagrams need the same current, the DAG contains one node and downstream amplitude roots reuse its value. Both LC report artifacts retain all physical flows and helicities. The topology-replay artifact selects $(2, 4, 1)$ at runtime and sums source helicities; the all-flow-union artifact selects one source-helicity assignment at runtime and sums every LC flow. Each generated artifact serializes its own stages and evaluator payloads; Rusticol loads it, executes `Runtime.evaluate()` inside the native core, and `BenchmarkRunner` records both the native repeated wall time and the sum of evaluator-call timings.

6.1 Dedicated $d\bar{d} \to Z+$ Gluon Performance

n	process	route	setup	selected flow, helicity sum gen [s]	selected flow, helicity sum wall [us/pt]	selected flow, helicity sum eval [us/pt]	all flows, fixed helicity gen [s]	all flows, fixed helicity wall [us/pt]	all flows, fixed helicity eval [us/pt]
1	d d~ > Z + (1-1)*g	reference	AmpliCol	7.19	0.0938	N/A	7.19	0.205	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	eager-DAG JIT O3	0.394 (x0.05)	0.477 (x5.09)	0.153 (x1.63)	0.296 (x0.04)	0.367 (x1.79)	0.0481 (x0.24)
2	d d~ > Z + (2-1)*g	reference	AmpliCol	7.44	0.447	N/A	7.44	0.363	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	eager-DAG JIT O3	0.408 (x0.05)	1.32 (x2.96)	0.823 (x1.84)	0.317 (x0.04)	0.621 (x1.71)	0.134 (x0.37)
3	d d~ > Z + (3-1)*g	reference	AmpliCol	7.5	1.3	N/A	7.5	0.854	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	eager-DAG JIT O3	0.42 (x0.06)	3.42 (x2.64)	2.76 (x2.13)	0.363 (x0.05)	1.16 (x1.36)	0.517 (x0.61)
4	d d~ > Z + (4-1)*g	reference	AmpliCol	7.67	4.17	N/A	7.67	3	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	eager-DAG JIT O3	0.484 (x0.06)	9.44 (x2.26)	8.55 (x2.05)	0.731 (x0.10)	3.06 (x1.02)	2.14 (x0.71)
5	d d~ > Z + (5-1)*g	reference	AmpliCol	8.09	12.3	N/A	8.09	15.4	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	eager-DAG JIT O3	0.602 (x0.07)	31 (x2.53)	29.8 (x2.43)	3.39 (x0.42)	16.5 (x1.07)	14 (x0.91)
6	d d~ > Z + (6-1)*g	reference	AmpliCol	8.87	34.1	N/A	8.87	97	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	eager-DAG JIT O3	1.23 (x0.14)	60.9 (x1.78)	59.1 (x1.73)	36.4 (x4.11)	165 (x1.71)	139 (x1.44)
7	d d~ > Z + (7-1)*g	reference	AmpliCol	10.6	92.3	N/A	10.6	845	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	eager-DAG JIT O3	6.7 (x0.63)	153 (x1.66)	150 (x1.63)	652 (x61.6)	2.39e3 (x2.83)	1.94e3 (x2.29)
8	d d~ > Z + (8-1)*g	reference	AmpliCol	16.3	228	N/A	16.3	2.13e4	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	eager-DAG JIT O3	N/A	N/A	N/A	skip	N/A	N/A
9	d d~ > Z + (9-1)*g	reference	AmpliCol	38.5	609	N/A	38.5	2.83e5	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol C++ O3	skip	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	eager-DAG JIT O3	N/A	N/A	N/A	skip	N/A	N/A

6.2 UFO-SM Dedicated $d\bar{d} \to Z+$ Gluon Performance

n	process	route	setup	selected flow, helicity sum					all flows, fixed helicity				
				gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]	gen [s]	vs blt-in	wall [us/pt]	vs blt-in	eval [us/pt]
1	d d~ > Z + (1-1)*g	reference	AmpliCol	7.21		0.125		N/A	7.21		0.127		N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1	d d~ > Z + (1-1)*g	Rusticol	eager-DAG JIT O3	0.627 (x0.09)	x1.59	0.498 (x3.99)	x1.04	0.169 (x1.36)	0.448 (x0.06)	x1.51	0.38 (x2.99)	x1.04	0.0533 (x0.42)
2	d d~ > Z + (2-1)*g	reference	AmpliCol	7.36		0.446		N/A	7.36		0.355		N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	d d~ > Z + (2-1)*g	Rusticol	eager-DAG JIT O3	0.762 (x0.10)	x1.87	1.33 (x2.99)	x1.01	0.831 (x1.86)	0.495 (x0.07)	x1.56	0.64 (x1.80)	x1.03	0.153 (x0.43)
3	d d~ > Z + (3-1)*g	reference	AmpliCol	7.68		1.3		N/A	7.68		0.929		N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	d d~ > Z + (3-1)*g	Rusticol	eager-DAG JIT O3	0.697 (x0.09)	x1.66	3.37 (x2.58)	x0.98	2.71 (x2.08)	0.512 (x0.07)	x1.41	1.14 (x1.23)	x0.99	0.507 (x0.55)
4	d d~ > Z + (4-1)*g	reference	AmpliCol	7.76		4.23		N/A	7.76		3.18		N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	d d~ > Z + (4-1)*g	Rusticol	eager-DAG JIT O3	0.763 (x0.10)	x1.58	9.35 (x2.21)	x0.99	8.49 (x2.01)	0.775 (x0.10)	x1.06	3.3 (x1.04)	x1.08	2.39 (x0.75)
5	d d~ > Z + (5-1)*g	reference	AmpliCol	8.26		12.3		N/A	8.26		15.4		N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	d d~ > Z + (5-1)*g	Rusticol	eager-DAG JIT O3	0.876 (x0.11)	x1.45	25.3 (x2.05)	x0.82	24.1 (x1.96)	3.45 (x0.42)	x1.02	17.5 (x1.14)	x1.06	15.1 (x0.98)
6	d d~ > Z + (6-1)*g	reference	AmpliCol	8.8		34.2		N/A	8.8		96.9		N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	d d~ > Z + (6-1)*g	Rusticol	eager-DAG JIT O3	1.36 (x0.16)	x1.11	67.8 (x1.98)	x1.11	66.2 (x1.93)	36.9 (x4.19)	x1.01	176 (x1.82)	x1.07	153 (x1.58)
7	d d~ > Z + (7-1)*g	reference	AmpliCol	10.4		91.3		N/A	10.4		1.6e3		N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	d d~ > Z + (7-1)*g	Rusticol	eager-DAG JIT O3	7.12 (x0.68)	x1.06	178 (x1.96)	x1.16	175 (x1.92)	644 (x61.9)	x0.99	2.69e3 (x1.69)	x1.13	2.23e3 (x1.40)
8	d d~ > Z + (8-1)*g	reference	AmpliCol	16.3		228		N/A	16.3		1.86e4		N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	d d~ > Z + (8-1)*g	Rusticol	eager-DAG JIT O3	N/A	N/A	N/A	N/A	N/A	skip	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	reference	AmpliCol	39.3		612		N/A	39.3		2.9e5		N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol C++ O3	skip	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	pyAmpliCol JIT O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	d d~ > Z + (9-1)*g	Rusticol	eager-DAG JIT O3	N/A	N/A	N/A	N/A	N/A	skip	N/A	N/A	N/A	N/A

Each LC workload block reports its own process generation in seconds and Rusticol-core wall and native execution-submetric runtime in microseconds per point: topology replay for selected-flow/helicity-sum, and all-flow union for all-flows/fixed-helicity. For compiled rows the submetric is generated stage/root evaluator calls; for eager rows it is inclusive eager execution. Parenthesized multipliers compare pyAmpliCol with AmpliCol; the UFO-SM "vs blt-in" columns compare the same UFO-SM and built-in-SM setup.

7 Colour-Singlet Model Ladders

The scalar-contact ladder retains $n = 2, \ldots, 8$ final scalars. It tests generic high-valence contact decomposition, identical-particle normalization, and source parity without Standard-Model-specific kernels. The scalar-gravity ladder retains $n = 2, \ldots, 4$ final gravitons and exercises spin-two wavefunctions, tensor propagators, and momentum-dependent vertices. Since all external particles in these fixtures are colour singlets, LC, NLC, and full colour reduce to the same unit colour contraction.

For these ladders, the relative-difference row is $|M_{\mathrm{f64}} - M_{\mathrm{hp}}| / \max(|M_{\mathrm{hp}}|, 10^{-300})$, where M_{hp} is the same point re-evaluated through the higher-precision diagnostic path. It is a numerical stability check, not a performance ratio.

7.1 Scalar-contact ladder

	scalar_0 scalar_0 > n*scalar_0						
metric	n=2	n=3	n=4	n=5	n=6	n=7	n=8
generation [s]	0.928	1.43	3.29	10.9	39.6	144	489
wall [μs/pt]	0.193	0.257	0.622	0.909	2.21	2.58	10.2
evaluator [μs/pt]	0.054	0.1	0.228	0.367	0.904	1.26	4.05
matrix element	0.5	0.167	0.0417	0.00833	0.00139	0.000198	2.48e-5
relative diff. vs hp	1.11e-16	1.67e-16	0	0	0	0	1.37e-16

7.2 Scalar-gravity ladder

	scalar_0 scalar_0 > n*graviton		
metric	n=2	n=3	n=4
generation [s]	2.23	28.2	272
wall [μs/pt]	2.68	1.41e3	1.35e4
evaluator [μs/pt]	2.11	1.32e3	1.3e4
matrix element	1.61e9	9.46e11	3.07e13
relative diff. vs hp	1.04e-15	4.13e-15	1.91e-14

8 Interpretation

This report establishes coverage, methodology, and the measured result set. Performance comparisons are meaningful only for a reviewed baseline produced from the same effective configuration on named hardware. Readers should treat every N/A entry as unavailable and should not interpolate, copy, or estimate a value from neighbouring multiplicities or model profiles.

The retained measurement records under `docs/results/` are authoritative for table completeness and provenance. The TeX inputs and this PDF are their publication presentation.

9 Worked $d\bar{d} \to Zgg$ Example

This section follows one process from its user-facing notation to the amplitude roots generated by `pyAmpliCol`. It is deliberately concrete: the counts and labels below are those produced by the built-in Standard Model for complete leading-colour (LC) coverage, not a schematic diagram count. The same process is also contained in the UFO-SM validation set.

9.1 External legs and conventions

The physical process is

$$d(p_1)\,\bar{d}(p_2) \longrightarrow Z(p_3)\,g(p_4)\,g(p_5), \qquad p_1 + p_2 = p_3 + p_4 + p_5. \tag{7}$$

The integers $1, \ldots, 5$ are *source labels*. They are stable labels of external particles in process order; they are neither current IDs nor colour-sector IDs. The DAG compiler crosses incoming particles once and then uses the all-outgoing momenta

$$q_1 = -p_1, \quad q_2 = -p_2, \quad q_i = p_i \ (i = 3, 4, 5), \qquad \sum_{i=1}^{5} q_i = 0. \tag{8}$$

Runtime clients nevertheless supply momenta in the physical order and sign convention of Eq. (7); Rusticol performs the two incoming sign changes when it fills momentum slots.

label	physical leg	DAG particle	DAG momentum	states	colour role
1	incoming d	outgoing $\bar{d}$	$q_1 = -p_1$	2	$\overline{\mathbf{3}}$, colour-line end
2	incoming $\bar{d}$	outgoing d	$q_2 = -p_2$	2	$\mathbf{3}$, colour-line start
3	outgoing Z	outgoing Z	$q_3 = p_3$	3	singlet
4	outgoing g	outgoing g	$q_4 = p_4$	2	adjoint insertion
5	outgoing g	outgoing g	$q_5 = p_5$	2	adjoint insertion

There are $2 \times 2 \times 3 \times 2 \times 2 = 48$ physical helicity configurations. The physics metadata retains all 48 stable helicity IDs, for example `h:+1,-1,+0,-1,+1`. Twenty-four configurations are proven structural zeros by the chiral source and kernel constraints and are returned as explicit zeros by resolved evaluation; only the nonzero representatives need amplitude roots.

At LC the singlet label 3 is absent from the coloured word. Complete coverage has the two physical flows

$$c_0 = (2, 4, 5, 1), \qquad c_1 = (2, 5, 4, 1), \tag{9}$$

published as the stable runtime IDs `flow:2,4,5,1` and `flow:2,5,4,1`. A user selects these strings. The small integer sector used while building a DAG is artifact-local implementation data and must not be used as a public flow identifier.

9.2 From diagrams to reusable currents

For a nonempty source subset I, define

$$q_I = \sum_{i \in I} q_i, \qquad m(I) = \sum_{i \in I} 2^{i-1}. \tag{10}$$

Thus $m(\{2, 4, 5\}) = 26$. A source wavefunction is a one-leg current $J_i^a = W_{t_i}^a(q_i, h_i)$, where W denotes the appropriate spinor, polarization vector, or scalar wavefunction. Composite currents obey the binary recursion

$$J_{t,\Omega}^a(I) = P_t{}^a{}_b(q_I) \sum_{\substack{I = A \dot\cup B \\ A,B \neq \varnothing}} \sum_{v\,:\,t_A t_B \to t} \kappa_v\, V_v{}^b{}_{cd}(q_A, q_B) J_{t_A,\Omega_A}^c(A) J_{t_B,\Omega_B}^d(B). \tag{11}$$

Here t is the particle state and Ω is the complete current identity described in Sec. 10. The sum contains only model, quantum-number, perturbative-order, and colour-order admissible combinations. The propagator P_t belongs to the output value slot; an unpropagated form is retained as well when a later closure needs it.

For one fixed source-state ancestry, representative terms along the quark line are

$$J_d(2,5) = P_d\, V_{dgd}\big[J_d(2), J_g(5)\big], \tag{12}$$

$$J_g(4,5) = P_g\, V_{ggg}\big[J_g(4), J_g(5)\big], \tag{13}$$

$$J_d(2,4,5) = P_d\Big(V_{dgd}\big[J_d(2,5), J_g(4)\big] + V_{dgd}\big[J_d(2,4), J_g(5)\big] + V_{dgd}\big[J_d(2), J_g(4,5)\big]\Big), \tag{14}$$

$$J_d(2,3,4,5) = P_d \sum_{A \dot\cup B = \{2,3,4,5\}} V\big[J(A), J(B)\big]. \tag{15}$$

Equation (15) includes all admissible positions of the Z emission and both gluon traversals. The notation suppresses chirality, helicity ancestry, couplings, and ordered-label data; those fields are never suppressed in the actual key. In particular, $J_d(2,5,4)$ and $J_d(2,4,5)$ are distinct ordered currents even though they carry the same unordered subset and momentum.

The final closure joins the endpoint current to source label 1:

$$\mathcal{A}^{(r)}_{h,c} = C^{(r)}_{\dot a a}\, J^{\dot a}_{\bar d}(1)\, J^{a}_{d}(2,3,4,5), \qquad \mathcal{A}_{h,c} = \sum_{r \in G(h,c)} \mathcal{A}^{(r)}_{h,c}. \tag{16}$$

The contraction $C^{(r)}$, root coupling, colour weight, and coherent-group identifier are model-owned records, rather than special cases in Rusticol.

9.3 The concrete graph

The current compiler creates the following deterministic topology. IDs are dense diagnostic indices in this artifact, not stable public selectors.

layer	curr.	attach.	kernel eval.	representative content
sources	11	0	0	two Weyl states on each quark leg, three Z states, and two states on each gluon; current IDs 0–10
$\|I\| = 2$	18	18	18	$J_d(2,3)$, $J_d(2,4)$, $J_d(2,5)$, and $J_g(4,5)$; current IDs 11–28
$\|I\| = 3$	40	80	72	mixed Zg quark currents and the two ordered $J_d(2,4,5)$ families; current IDs 29–68
$\|I\| = 4$	48	144	120	endpoint currents with stored orders $(2,5,4,3)$ and $(2,4,5,3)$; current IDs 69–116
amplitude	—	48 roots	48 outputs	24 nonzero helicities for each physical colour flow
total	117	242	210	no auxiliary current is needed for this process

An *attachment* says that one interaction contributes to one target current. It need not imply a fresh kernel call. For example, the source quark current combined with $J_g(4,5)$ can feed both ordered three-leg targets. The two attachments share one verified `evaluation_group_id`; a sign or scalar `evaluation_factor` is applied at each target. This is why the three- and four-leg layers contain 224 attachments but only 192 distinct kernel evaluations. The resulting sharing is stronger than merely caching a list of Feynman diagrams.

9.4 Coverage, execution layout, and reproduction

Normal public generation materializes both flows and complete helicity metadata. Runtime selection then uses the stable physical IDs. For the report's selected-flow/helicity-sum workload, the topology-replay layout maps the source-label order $(2,5,4,1,3)$ to the coloured traversal $(2,5,4,1)$; the singlet 3 remains source-order provenance. The all-flow-union layout retains the same two public flows but constructs their shared recurrence directly. Section 11 gives the two layout contracts. Neither report artifact fixes this flow or a helicity during generation.

A deliberately selected-coverage diagnostic artifact can restrict this process to one flow. Such an artifact has 65 currents, 90 interactions, and 24 roots, but those internal counts and its artifact-local sector ID are not substitutes for the public flow ID and are not used for reusable report timings.

name space	**example**	**meaning**
source label	4	fourth particle in the user process
all-outgoing leg	$g(q_4)$	crossed DAG input with $q_4 = p_4$
internal sector ID	0	artifact-local generation index
physical flow ID	`flow:2,5,4,1`	stable runtime selector
current ID	69	dense slot for one complete current identity
value slot	propagated current 69	runtime storage variant and component range
root/output	one (h, c) contribution	complex amplitude before squaring

For the built-in compatibility model, Rusticol applies

$$|\mathcal{M}|^2 = \mathcal{N} \sum_{h,c} w_{h,c} \left| \sum_{r \in G(h,c)} \mathcal{A}^{(r)}_{h,c} \right|^2, \qquad \mathcal{N} = \frac{27\,(4\pi\alpha_s)^2 (8\pi\alpha_{\mathrm{EW}})}{36 \times 2}. \tag{17}$$

The denominator averages the two incoming colour/spin state spaces (6×6); the final factor $2 = 2!$ accounts for identical gluons. The physics manifest records the factors, coupling powers, coherent groups, and symmetry weights separately, so changing runtime model parameters refreshes the normalization without rebuilding evaluator code.

The installed examples provide a checkout-independent public route:

```
pyamplicol examples copy ./pyamplicol-examples
cd pyamplicol-examples
pyamplicol generate_pp_zjj_from_ufo_sm.toml
pyamplicol inspect artifacts/pp_zjj --process 'd d~ > z g g'
pyamplicol profile artifacts/pp_zjj --process 'd d~ > z g g' \
  --target-runtime 1 --batch-size 128
```

The generated artifact contains a process set; the exact expression is therefore the clearest selector. In Python, the same complete artifact exposes both the summed and resolved contracts:

```
from pyamplicol import Runtime

runtime = Runtime.load("artifacts/pp_zjj", process="d d~ > z g g")
print(runtime.physics.color_flow_ids)
total = runtime.evaluate(momenta)
resolved = runtime.evaluate_resolved(
    momenta, color_flows=["flow:2,5,4,1"]
)
```

Public TOML and CLI generation retain complete flow and helicity coverage for this workload. Deliberately selected coverage is an expert diagnostic and is not used for the report measurements. Both coverage forms use the same public process metadata and numerical reduction.

10 The $d\bar{d} \to Zgg$ DAG and Rusticol

The previous section described the physics graph. This section gives the exact mapping from that graph to compiled multi-output evaluators and then to the Rusticol runtime. The main invariant is simple: generation owns physics identity and symbolic lowering; Rusticol owns a validated numerical schedule of slots, stages, roots, and reductions.

10.1 What identifies a current?

The immutable `CurrentIndex` key is defined in `src/pyamplicol/generation/dag_types.py`. Two contributions share a current if and only if every field in

$$\Omega(J) = \big(t,\ m_I,\ I,\ H,\ \chi,\ s,\ F,\ Q,\ C,\ m_p,\ O,\ a,\ \pi_I\big) \tag{18}$$

is equal. There is no process-family or diagram label in this key.

field	example	role
`particle_id` t	1	off-shell particle carried by the current
`external_mask` m_I	26	bit mask of source subset $I = \{2, 4, 5\}$
`external_labels` I	$(2, 4, 5)$	sorted, unordered source-label set
`helicity_ancestry` H	source-state bit set	exact one-leg state choices feeding the current
`chirality` χ	$+1$ or -1	Weyl-flow branch when applicable
`spin_state` s	scalar or tuple	local spin/polarization state beyond chirality
`flavour_flow` F	model tuple	conserved flavour-line identity
`quantum_number_flow` Q	name/expression pairs	exact model-derived additive charges
`color_state` C	LC state	accuracy, internal sector, line groups, and basis keys
`momentum_mask` m_p	26	momentum slot used by kernels and propagator
`coupling_orders` O	`QCD=2`	accumulated perturbative order envelope
`auxiliary_kind` a	`None`	proof-backed contact-decomposition species, if present
`ordered_external_labels` π_I	$(2, 5, 4)$	colour-sensitive traversal within the same subset

Both the unordered labels and their order are intentional: the former fixes the momentum and subset, while the latter prevents two colour-ordered currents from being merged incorrectly. Conversely, exact equality permits reuse across every diagram and amplitude root that reaches the same state. The `_CurrentTable.add_or_get()` operation in `src/pyamplicol/generation/dag_table.py` is the sole identity-deduplication point. The resulting `current_id` is only the dense array index of that canonical node.

LC generation adds a second, complementary reuse mechanism. Internal currents are built in a shared-ordering state and their ordered labels and basis keys record which flow segments they can serve. Physical sector IDs are restored only when a colour line closes into an amplitude root. The two roots in Eq. (9) therefore share sources, electroweak subcurrents, the $J_g(4, 5)$ family, and every other exactly equal current, instead of cloning a complete recursion per flow.

Each one-leg state receives its own bit when sources are constructed; composing currents takes the bitwise OR. Thus H identifies concrete source-state choices, rather than merely the set of external labels.

Finally, `evaluation_group_id` addresses reuse that current identity alone cannot express. If one verified kernel value contributes, possibly with different scalar `evaluation_factors`, to several target currents, the symbolic stage computes it once. For the worked graph this reduces 242 target attachments to 210 kernel evaluations. These three notions must remain distinct:

$$\underbrace{\text{same current key}}_{\text{one stored node}},\qquad \underbrace{\text{same evaluation group}}_{\text{one symbolic kernel value}},\qquad \underbrace{\text{same physical flow}}_{\text{one public output axis member}}\ . \tag{19}$$

10.2 Topological stages and value slots

The compiler sweeps proper source subsets in increasing cardinality. An interaction whose result covers k source labels belongs to recursion stage k. No evaluator can read a current from its own or a later stage, so the schedule is acyclic by construction. For this example,

$$\boxed{11 \text{ sources}} \longrightarrow \boxed{|I| = 2:\ 18} \longrightarrow \boxed{|I| = 3:\ 40} \longrightarrow \boxed{|I| = 4:\ 48} \longrightarrow \boxed{48 \text{ amplitude roots}}. \tag{20}$$

For one concrete source-state path, the same schedule can be read as

$$E_2: \quad C_{21} = \mathcal{P}_{21} K_{10}(C_2, C_9), \qquad C_{25} = \mathcal{P}_{25} K_{14}(C_7, C_9), \tag{21}$$

$$E_3: \quad C_{53} = \mathcal{P}_{53}\big[K_{66}(C_{21}, C_7) + K_{82}(C_2, C_{25}) + \cdots\big], \tag{22}$$

$$E_4: \quad C_{69} = \mathcal{P}_{69}\big[K_{90}(C_{53}, C_4) + \cdots\big], \tag{23}$$

$$E_{\text{root}}: \quad \mathcal{A}_0 = \mathcal{C}_0(C_{69}, C_1). \tag{24}$$

Here C_2 is the label-2 quark source with helicity -1, C_7 and C_9 are the negative-helicity gluon sources on labels 4 and 5, C_4 is the negative-helicity Z source, and C_1 is the endpoint antiquark source. The omitted terms are other admissible partitions contributing coherently to the same target. The subscripts on K are evaluation-group IDs, not current or interaction IDs.

```
sources: C2=d(2,-), C7=g(4,-), C9=g(5,-), C4=Z(3,-), C1=dbar(1,+)
E2: I10: C2 + C9 -> C21 [G10] I14: C7 + C9 -> C25 [G14]
E3: I66: C21 + C7 -> C53 [G66] I83: C2 + C25 -> C53 [G82]
    I82: C2 + C25 -> C61 [G82] (one kernel value, two targets)
E4: I98: C53 + C4 -> C69 [G90]
root: R0: contract(C69,C1) -> A0
```

This adjacency-list excerpt is diagnostic but exact for the current graph. It also exposes why the namespaces cannot be conflated: C_{53} is a canonical current node, I_{83} is one attachment to it, G_{82} identifies a reusable symbolic evaluation, and R_0 is an amplitude output. Value-slot IDs are a further runtime namespace assigning propagated or unpropagated component storage to currents. Internal LC sector IDs annotate root construction; public IDs such as `flow:2,5,4,1` label the resolved physics axis.

The stage records are assembled by `build_runtime_schema()` in `src/pyamplicol/generation/runtime_schema.py`. They list input/output current IDs, value-slot IDs, momentum slots, interaction IDs, and both attachment and unique-evaluation counts.

A current and its stored value are not identical notions. Depending on its consumers, one current can have a `source`, `unpropagated`, or `propagated` value slot. Each slot owns a contiguous component range; Weyl currents have two complex components and vector currents have four in the worked model. Keeping variants explicit prevents the runtime from silently applying a propagator twice and lets the amplitude closure request precisely the form it needs.

The complete schema for this graph has 256 complex value components, 13 four-vector momentum slots (52 real components), and 16 real model-parameter slots. A conceptual flattened state row therefore has

$$x = \big[x_{\text{value}} \mid x_{\text{momentum}} \mid x_{\text{model}}\big], \qquad |x| = 256 + 52 + 16 = 324. \tag{25}$$

No compiled stage accepts all 324 entries. Stage-local layout is mandatory: for stage k, a map L_k selects only the current components, momentum components, and model parameters actually used there,

$$x_k = L_k x, \qquad y_k = E_k(x_k), \qquad x[S_k] \leftarrow y_k. \tag{26}$$

The construction is in `src/pyamplicol/generation/stage_parameters.py`; Rusticol stores the same local-to-global indices in the execution manifest. Contiguous index runs are precomputed, so most packing and output assignment becomes slice copies rather than scalar lookup loops.

10.3 What Symbolica and SymJIT compile

`build_generic_stage_compiler_blueprint()` in `src/pyamplicol/generation/stage_planning.py` creates one multi-output symbolic blueprint per recursion stage and one for the amplitude roots. For a target current j, its output expression is

$$y_j = \mathcal{P}_j \left[\sum_{e:\, \text{target}(e)=j} f_e \, K_{g(e)}(x_k) \right], \tag{27}$$

where $K_{g(e)}$ is cached by evaluation-group ID, f_e is the attachment factor, and $\mathcal{P}_j$ is the output propagator or identity. Component expressions, couplings, momentum dependence, and propagators come from the selected model; the stage compiler does not know that this example contains a Z or a gluon.

Symbolica optimizes the whole multi-output expression of one stage: common subexpressions, Horner schemes, and common-pair elimination can therefore cross current-component boundaries inside that stage. With the default JIT backend, SymJIT O3 then compiles a batched complex-f64 application. The direct application capability is recorded as `symjit.application.complex-f64.v1`; Rusticol can load and execute this f64 payload without importing Symbolica. Retained Symbolica evaluator state is used for higher-precision Python requests.

Output chunking is a compilation boundary, not a physics boundary. If a stage has more than the configured 512 scalar outputs, it is split into multi-output chunks sharing the same local input vector. A fanout-aware ordering in `src/pyamplicol/generation/stage_planning.py` places outputs with common evaluation groups near one another, reducing duplicated work when a group would otherwise cross chunk boundaries. At runtime the local input is packed once per stage and batch, each compiled chunk consumes that same packed buffer, and the concatenated outputs are scattered to their recorded slots. Dependencies are still cut only between recursion stages.

Compiled generation streams these stages. Once a blueprint has been compiled and written, its Symbolica expressions can be released before the next stage is built. This bounds peak expression memory without changing the DAG or the runtime schedule. The relevant writer path is `src/pyamplicol/generation/stage_artifacts.py`, while backend settings and payload construction live in `src/pyamplicol/evaluators/symbolica_compile.py`.

10.4 Artifact records and Rusticol execution

For the compiled route described in this section, the root `artifact.json` is a process-artifact schema-v3 manifest. It binds global model/configuration data and payload hashes to process-local records. The files relevant to one concrete process are:

record	runtime meaning
`processes/evaluators.json`	evaluator capabilities and payload locations
`processes/<id>/execution.json`	value/momentum/model layout, sources, ordered stages, chunks, roots, and reduction schedule
`processes/<id>/physics.json`	external particles, physical helicity and flow IDs, coverage, symmetry expansion, selectors, and normalization
evaluator applications/libraries	target-specific compiled numerical kernels
`API/`	validation point and Python, Rust, C++, and Fortran examples

The complete `CurrentIndex` is a generation contract. The execution projection serializes only the fields needed to validate and run its slots; ordered-label proofs and other generation-only evidence need not burden the hot runtime. By contrast, physical helicity and flow metadata remains explicit because it defines the public resolved axes and selector semantics.

Rusticol's f64 path is implemented in `rust/crates/rusticol-core/src/engine/evaluation.rs`. For a batch of points, one execution is:

1. reuse and clear the state scratch buffer;
2. fill source wavefunctions in their source value slots;
3. form all recorded all-outgoing momentum sums and copy current model parameter values into the global state;
4. for every recursion stage, pack x_k, call all evaluator chunks, and assign y_k to the global value slots;
5. pack and call the amplitude-root evaluator;
6. sum roots coherently, contract colour, expand exact physical symmetry representatives when resolved output is requested, and apply normalization.

The stage and amplitude implementations reuse their parameter and output scratch buffers across calls. The global model-parameter vector is mutable at runtime; an atomic update validates all supplied values, refreshes derived parameters and masses, and recomputes the factor in Eq. (17). No mass, width, or coupling is baked into the orchestration schedule merely because it is constant in one run card.

For LC resolved evaluation, the public tensor has shape

$$R_{n,h,c} = |\mathcal{M}_{h,c}(p_n)|^2, \qquad \text{shape}(R) = (N_{\text{point}}, N_{\text{helicity}}, N_{\text{flow}}). \tag{28}$$

For this process the final two dimensions have lengths 48 and 2. Structural zeros occupy their declared entries. `evaluate_resolved()` can select stable helicity/flow IDs and materializes these components; its `total()` is their explicit sum. `evaluate()` follows the optimized summed path directly and need not allocate the resolved tensor. Neither interface exposes internal current, slot, or sector IDs.

10.5 The two runtime timing boundaries

The profiler deliberately distinguishes orchestration from compiled evaluator calls. Its reported f64 wall sample starts in the native Rusticol method after the caller-language buffer conversion and repeatedly executes the ordinary core path through final reduction. It therefore includes Rust-side input batch preparation, source and momentum construction, stage-local packing, chunk calls, output assignment, root packing/call, and reduction, but excludes Python/NumPy, C++, Fortran, or Rust caller-side packing.

The evaluator sum is narrower:

$$t_{\text{eval}} = \sum_k t\big(E_k\big) + t\big(E_{\text{root}}\big), \qquad t_{\text{core}} = t_{\text{eval}} + t_{\text{source}} + t_{\text{momentum/model}} + t_{\text{pack}} + t_{\text{assign}} + t_{\text{reduce}} + t_{\text{batch}}. \tag{29}$$

provenance marker	**quantity**
`runtime_core_repeated_wall_time`	native repeated total path, normalized per point; common metric for all language APIs
`runtime_profile_core_evaluator_call_time`	sum of recursion-stage and amplitude-root evaluator call times only
native component profile	source fill, momentum/model setup, per-stage input pack, evaluator call and output assignment, root pack/call, and reduction

These boundaries make a wall/evaluator difference interpretable as work inside Rusticol rather than Python probing overhead. They also make Python, Rust, C++, and Fortran measurements comparable when they call the same f64 artifact. The CLI calibration, uncertainty estimate, progress display, and interruptible sampling policy are implemented in `src/pyamplicol/benchmarking.py`; the native timers themselves are in `rust/crates/rusticol-core/src/engine/native_runtime.rs` and `rust/crates/rusticol-core/src/engine/evaluation.rs`.

11 Reusable Leading-Colour Execution Layouts

Leading-colour coverage and leading-colour execution layout are separate contracts. Let $\mathcal{H}$ be the retained physical helicity IDs and let $\mathcal{F}$ be the retained physical colour-flow IDs. A complete LC artifact represents

$$\mathcal{C}_{\mathrm{LC}} = \mathcal{H} \times \mathcal{F} \tag{30}$$

and can therefore expose every resolved component $\mathcal{A}_{hf}$, independently of how its recurrence is scheduled. For runtime-selected subsets $S_H \subseteq \mathcal{H}$ and $S_F \subseteq \mathcal{F}$, Rusticol computes

$$\mathcal{W}(S_H, S_F) = \mathcal{N} \sum_{h \in S_H} \sum_{f \in S_F} w_{hf} \left| \sum_{r \in G(h,f)} \mathcal{A}_{hf}^{(r)} \right|^2 . \tag{31}$$

Here $G(h, f)$ is one coherent root group, w_{hf} contains the exact symmetry and LC weights, and $\mathcal{N}$ is the process normalization. A layout changes the representation and order used to evaluate these terms; it does not change $\mathcal{H}$, $\mathcal{F}$, or Eq. (31).

11.1 Two complete-coverage layouts

`pyAmpliCol` provides two LC layouts because the two principal workloads expose different opportunities for current sharing.

property	`topology-replay`	`all-flow-union`
Primary workload	one runtime-selected flow, summed helicities	all flows, one runtime-selected helicity
Retained physics	every physical flow and helicity	every physical flow and helicity
Main reuse	exact flow-topology representatives, label permutations, signs, and multiplicities	currents, interactions, and closures shared directly across the union of physical flows
Compiled lane	representative fused stage recurrences replayed by Rusticol	fused cross-flow union stages selected by runtime helicity
Eager lane	prepared-kernel invocations scheduled by replay class	prepared-kernel invocations lowered from the same cross-flow union
Default	yes	explicit opt-in

In the topology-replay layout, exact certificates partition physical flows into recurrence classes. If $\rho(f)$ is the representative of the class containing f, the artifact records a source-label permutation π_f, an exact sign or factor s_f, and its reduction weight. Schematically,

$$\mathcal{A}_{hf}(p) = s_f \, \mathcal{A}_{\pi_f(h),\rho(f)}(\pi_f(p)) \, . \tag{32}$$

Rusticol maps the requested flow to its representative, groups compatible point rows, evaluates contiguous backend batches, and scatters the weighted result to the public flow ID. This avoids materializing one independent DAG for every physical ordering and is the layout used for the single-flow/helicity-sum benchmark.

The all-flow-union layout instead constructs the complete cross-flow current DAG before evaluator lowering. If D_{hf} is the dependency graph of one resolved component, its graph is the exact quotient

$$D_{\mathrm{union}} = \left(\bigcup_{h \in \mathcal{H},\, f \in \mathcal{F}} D_{hf} \right) / \sim , \tag{33}$$

where $\sim$ identifies currents, kernel evaluations, attachments, and closures whose canonical contracts prove them equal up to stored factors. For a selected helicity, one sweep of this union shares common intermediate currents across all requested flows. This is the layout used for the all-flows/single-helicity benchmark. The artifact does not also carry a second topology-replay graph or a generation-fixed-helicity graph.

Both layouts are available with compiled and eager execution. Compiled mode lowers the chosen graph to process-specific stage evaluators. Eager mode lowers it to invocation tables over the prepared model kernels. Thus `execution_mode` selects how numerical kernels are supplied, while `lc_flow_layout` selects how complete LC recurrence work is shared.

11.2 Runtime selectors

Public selectors always use the stable IDs stored in the physics manifest, such as `flow:2,5,4,1` and `h:-1,+1,+0,-1,+1`. Batch-global `color_flows` and `helicities` select rectangular subsets of Eq. (30). The per-point `color_flow_by_point` and `helicity_by_point` inputs select one component on the corresponding axis for each point. A global and a per-point selector are mutually exclusive on the same axis. Omitting an axis sums every retained component on that axis.

Rusticol stably groups per-point selector rows before numerical evaluation so that equal selectors form contiguous SIMD-compatible packets. An already grouped input remains in order and does not pay for a redundant sort. This selector planning is a runtime operation; it does not reduce the generated coverage of either layout. Deliberately generation-sliced artifacts remain a separate expert facility and, by construction, can select only components that were retained during their generation. The all-flow-union layout requires complete flow and helicity coverage and therefore rejects such generation slices.

The layout is selected in a run card with

```
[color]
accuracy = "lc"
lc_flow_layout = "all-flow-union"
```

or with `-lc-flow-layout all-flow-union`. Omitting the option keeps the `topology-replay` default.

11.3 Why the report has two generation times

The two layouts retain the same physical component set but build different graphs, evaluator payloads, and runtime schedules. Their process-generation times are consequently distinct observables,

$$T_{\text{gen}}^{\text{replay}} \quad \text{and} \quad T_{\text{gen}}^{\text{union}}, \tag{34}$$

and neither value is copied into the other workload's table slot. The LC selected-flow/helicity-sum columns use $T_{\text{gen}}^{\text{replay}}$; the all-flows/single-helicity columns use $T_{\text{gen}}^{\text{union}}$. Original `AmpliCol` produces one table-driven library for both workloads, so its one generation time remains the denominator of both `pyAmpliCol`/`AmpliCol` ratios. In eager-versus-compiled tables, each eager generation is compared with the compiled generation using the same LC layout. Prepared-model creation is a reusable model-level operation and remains outside either eager process-generation timer.

11.4 NLC and full colour

The layout option applies only to LC. NLC and full-colour artifacts already construct one complete amplitude basis and contract it with a sparse colour metric,

$$\mathcal{W}_{\text{contracted}} = \mathcal{N} \sum_{h} \sum_{a,b} \mathcal{A}_{ha}^{*}\, C_{ab}\, \mathcal{A}_{hb}. \tag{35}$$

They do not expose the physical LC-flow axis whose alternative scheduling motivates topology replay and the all-flow union. Configuration therefore rejects `all-flow-union` for NLC and full colour. Their generation, contraction, cache, and runtime contracts are unchanged by this choice.

12 Prepared-Model Eager DAG Execution

`pyAmpliCol` provides two execution contracts. The default *compiled* mode turns a complete process stage into one or more process-specific evaluator applications. The opt-in *eager* mode compiles the model-local vertex, current-finalization, closure, and parameter kernels once, then represents a process by compact invocation tables. Rusticol interprets those tables in its numeric core. The modes use the same physical DAG, normalization, colour reduction, selectors, and public APIs, but deliberately place the compilation boundary at different levels:

	Compiled mode	**Eager mode**
Compiled unit	A process-stage expression containing many current recurrences	One canonical model-local kernel evaluation class
Process generation	Builds and compiles process-specific stage applications	Rust lowers proven DAG columns to invocation, finalization, closure, reduction, and selector tables
Runtime scheduling	Calls a small number of wide process-stage applications	Gathers packets, calls reusable kernels, scatters contributions, and finalizes currents
Coverage contract	Complete artifacts support runtime selectors; selected-coverage artifacts may specialize	The same complete-coverage and selected-coverage contracts
Default	Yes	No; selected with `execution_mode = "eager"`

Eager execution is not a replacement for compiled execution. It is a second lane for applications where rapid process generation, reusable model kernels, or runtime selector flexibility matters more than maximal process-wide kernel fusion. No eager branch is present in the compiled lane's inner loop, and the compiled artifact contract remains unchanged.

12.1 Prepared model and process plan

A prepared model bundle separates model work from process work. Schematically, the bundle is

$$\mathcal{B}_{M,b,o,a} = \left(\mathcal{I}_M,\ \mathcal{E}_M,\ \{K_\kappa^{(b,o,a)}\}_{\kappa\in\mathcal{K}_M},\ \mathcal{P}_M\right), \tag{36}$$

where $\mathcal{I}_M$ is the portable compiled-model IR, $\mathcal{E}_M$ retains exact expressions, K_κ is one prepared kernel for canonical evaluation class κ, and $\mathcal{P}_M$ contains ABI and provenance metadata. The labels b, o, a identify backend, optimization settings, and architecture class. A bundle contains exactly one prepared JIT, C++, or ASM backend pack.

Eager process generation keeps symbolic and numerical responsibilities separate. Python performs the Symbolica algebra, physics proofs, prepared-kernel resolution, and extraction of the proven `GenericDAG`. It then fills fixed-width columnar arrays for currents, interactions, roots, kernel references, permutations, aliases, selector proofs, and reductions. Rust validates those columns and lowers them directly to

$$\mathcal{Q}_P = \left(\{\mathsf{Inv}_s, \mathsf{Att}_s, \mathsf{Fin}_s\}_{s=1}^{S},\ \mathsf{Close},\ \mathsf{Reduce},\ \mathsf{Select}\right). \tag{37}$$

Here Inv_s identifies kernel calls in recursion stage s, Att_s maps their outputs to current components, Fin_s applies the current's finalization or propagator once, Close builds amplitudes, and Reduce performs the physical colour/helicity contraction. Select stores dependency domains for runtime selectors. The Rust lowering assigns slots, interns selector domains, groups invocations, schedules attachments and finalizations, constructs closures and coherent reductions, validates the resulting plan, and writes the indexed binary runtime layout. It does not return expanded Python row objects or construct a large JSON runtime schema.

No backend evaluator is built or compiled during this process step. Invocation rows refer to kernels already present in $\mathcal{B}$, and the referenced prepared applications are copied into the standalone process artifact. The cost of preparing $\mathcal{B}$ is therefore a separate model-level quantity, not part of eager process-generation time.

Prepared kernels are derived from the same verified oriented-kernel catalog used by the model-generic UFO lowering. The built-in Standard Model emits that same catalog, while its explicitly Standard-Model-specific physics remains under `src/pyamplicol/models/builtin/`. Couplings, colour coefficients, exchange signs, input permutations, and proof factors are invocation metadata, not distinct compiled applications. Existing proof-gated auxiliary-current decompositions continue to reduce supported higher-point interactions to the same local kernel classes described here and in the UFO-support section below.

12.2 Stage execution

Let $V_{c\alpha p}$ be component α of current slot c at phase-space point p. An invocation row r stores a kernel identifier $\kappa(r)$, input slots and component permutations, momentum slots, coupling references, an exact factor w_r, and an output attachment. For one stage, Rusticol first forms unpropagated current components

$$U^{(s)}_{c\alpha p} = \sum_{r\in\mathcal{R}_s(c,\alpha)} w_r \left[K_{\kappa(r)}\left(G_r(V_{\cdot\cdot p}),\ q_r(p),\ \theta\right)\right]_{\alpha(r)}. \tag{38}$$

The gather map G_r is fixed by the invocation table. Contributions are accumulated before current finalization, so a current with momentum Q_c is materialized exactly once:

$$V^{(s)}_{c\beta p} = \left[F_{\phi(c)}\left(U^{(s)}_{c\cdot p}, Q_c(p), \theta\right)\right]_{\beta}. \tag{39}$$

The finalization class $\phi(c)$ may be an identity, a propagator, or another model-proved current transformation. This ordering prevents a propagator from being applied once per contributing vertex and preserves the compiled-mode normalization convention.

After the last recursion stage, closure rows produce amplitude components,

$$A_{h\gamma p} = \sum_{r\in\mathcal{C}(h,\gamma)} w^{\mathrm{cl}}_r K^{\mathrm{cl}}_{\kappa(r)}\left(G^{\mathrm{cl}}_r(V_{\cdot\cdot p}), q_r(p), \theta\right), \tag{40}$$

and the existing LC or contracted-colour reduction computes the requested total or resolved observable. In a contracted basis this has the schematic form

$$\mathcal{M}_p = \mathcal{N}\sum_h \sum_{\gamma,\delta} A_{h\gamma p}\, C_{\gamma\delta}\, A^*_{h\delta p}, \tag{41}$$

with the same colour matrix C, symmetry factors, initial-state average, and coupling normalization as compiled mode.

The Rust implementation is deliberately separate from compiled execution. Plan loading and validation are in `rust/crates/rusticol-core/src/eager_runtime/plan.rs`; the allocation-free numeric loop is in `eager_runtime/execute.rs`; runtime ownership and workspace reuse are in `eager_runtime/runtime.rs`. The artifact-facing integration is isolated under `rusticol-core/src/engine/eager_*.rs`.

12.3 Packetization, tiling, and allocation discipline

Rusticol does not call a backend once per DAG edge. Invocation rows are packetized by kernel class and compatible mapping. For a tile of T points, the innermost layout keeps points contiguous,

$$X[\text{argument}][\text{invocation}][\text{point}], \tag{42}$$

so a SymJIT application can auto-vectorize across points. C++ and ASM packs receive the same batched packet but remain scalar across rows; no SIMD claim is made for those backends.

Current storage is component-major with points contiguous. Gather buffers, kernel outputs, current storage, selector masks, closure storage, and reduction workspace are allocated at load or selector change and then reused. After warm-up, `evaluate_into` performs no heap allocation in the Rust numeric core. Large batches are tiled using

$$T_{\mathrm{eff}} = \min\left(T_{\mathrm{requested}}, \left\lfloor \frac{W_{\mathrm{limit}}}{W_{\mathrm{per\ point}}} \right\rfloor\right), \tag{43}$$

subject to a minimum valid packet. The public defaults are a requested tile size of 1024 points and a 256 MiB workspace. This keeps memory bounded for arbitrarily large Monte-Carlo batches without placing Python or NumPy packing inside the reported Rusticol wall time.

Eager profiling separates initialization, gather, kernel calls, invocation scatter, finalization, finalization scatter, closure, reduction, and copy-out. Those markers are returned by `pyamplicol profile` and the same native runtime profile used by Python, C11, C++17, Fortran 2008, and Rust 2021 clients. The report's eager `eval` field is the inclusive eager execution lane, not kernel-call time alone; the detailed profile separates its constituent phases.

12.4 Runtime selectors and complete LC coverage

For a selector σ, eager lowering stores a dependency domain $D(x)$ for each invocation, attachment, finalization, and closure row x. The active schedule is

$$\mathcal{Q}_P(\sigma) = \{x \in \mathcal{Q}_P : D(x) \cap \sigma \neq \varnothing\}, \tag{44}$$

including all transitive current dependencies. Masks and packet schedules are rebuilt only when a selector changes, not at every phase-space point.

This distinction is important for LC timing. Every *complete* eager LC artifact contains each physical flow and generated helicity component. A caller may select any one flow for a helicity sum, one helicity for all flows, or any supported subset at runtime. The flow identifier, for example `flow:3,1,2,5,6,7,4`, is therefore a runtime choice rather than a flow hard-wired during process generation. A deliberately selected-coverage eager artifact remains limited to the components that were generated.

Compiled mode exposes the same complete-coverage runtime contract. Coverage does not, however, prescribe one execution graph. As described in Sec. 11, both execution modes provide topology-replay and all-flow-union layouts. The report uses topology replay for single-flow/helicity-sum and the all-flow union for all-flows/single-helicity, with separate process-generation times. Either artifact can choose any retained flow or helicity without regeneration. Deliberately specialized artifacts remain useful as optimization baselines, but are not the reusable report workloads.

12.5 Artifact, precision, and portability contracts

An eager process artifact is standalone. It copies each referenced prepared kernel exactly once under the artifact root and never depends on the original prepared-model path. Fixed-width little-endian tables use 32-bit identifiers and 64-bit offsets/counts. Plan-v3 artifacts use runtime kind `pyamplicol-runtime-eager-execution`, capability `rusticol.eager-runtime-layout.complex-f64.v1`, and four complementary records:

- a small `execution.json` summary containing identity, ABI, provenance, counts, digests, options, and inspection statistics;
- a compact `physics.json` containing public process axes, coverage, selector metadata, parameters, normalization, and proof summaries;
- `processes/<id>/eager-runtime.pacbin`, containing the process-local invocation, slot, selector, closure, reduction, and proof tables; and
- the artifact-root `evaluators.pacbin`, containing indexed prepared evaluators and their exact states.

High-cardinality runtime data is not duplicated in JSON. Rusticol authenticates the artifact and PACBIN indices, memory-maps the containers, and decodes the eager layout into a validated owned execution plan. Normal `inspect` operations use the summary metadata rather than scanning every table. The loader rejects malformed offsets, unknown kernel identifiers, incompatible ABIs, and target mismatches before entering native evaluation.

This representation reduces Python object construction, JSON serialization, filesystem entry count, artifact footprint, peak generation memory, and cold load work. It does not change the eager numerical schedule: topology replay, LC all-flow union, NLC/full contraction, aliases, structural zeros, and global or per-point selectors retain their existing semantics. Both LC layouts use the same Rust lowering and compact container contract. Exact Python execution reads the same indexed plan and evaluator states. Target-native C++ and ASM libraries remain separate payloads.

Plan-v3 is the eager artifact compatibility boundary. Earlier eager artifacts use an expanded runtime representation and must be regenerated; the loader identifies that capability before opening the old high-cardinality payload and reports the required regeneration. This requirement is limited to eager artifacts and does not change compiled artifacts or prepared-model bundles.

JIT packs store SymJIT application state rather than final machine code. With the SymJIT storage v3 contract, register-allocation state is scoped to an architecture class. Wheels therefore carry separate `x86_64` and `aarch64` built-in-SM JIT O3 packs and select the host pack before DAG construction. Transfer is tested across operating systems within a matching architecture class; cross-architecture loading is rejected. C++ and ASM packs are target-native.

Symbolica is required during eager generation and for non-f64 exact execution. After generation, saved JIT applications support Symbolica-free f64 execution, including multicore use. Exact expressions remain in the artifact, and `src/pyamplicol/runtime/eager_exact/` executes the same plan for double-double and arbitrary precision. Thus the execution-mode choice does not change the public precision contract.

12.6 Performance interpretation and scaling

Eager process-generation timing begins with an already prepared model bundle and ends with an authenticated, loadable process artifact. It includes Python proof/DAG extraction, transfer-column construction, Rust plan lowering, compact container writing, and post-build validation. Prepared-model creation is reported separately. Useful comparisons therefore report process generation, prepared-model creation, artifact size, peak resident memory, authenticated cold load, and first-use break-even as distinct quantities.

Rust table lowering and high-cardinality serialization handle the part of eager generation that grows with the process DAG. Compact PACBIN storage also avoids materializing equivalent expanded Python and JSON objects. The resulting benefit depends on current count, selector-domain complexity, colour contraction, and proof metadata; it is not represented by one universal speedup. Published process and Z-ladder tables provide the measured values under the benchmark contract defined in the main methodology section.

For runtime, eager mode avoids process-specific backend compilation but pays explicit gather, scatter, finalization, and reusable-kernel dispatch. SymJIT amortizes those costs over point batches through internal SIMD. Selector measurements distinguish homogeneous, already grouped, alternating, and random per-point selections because regrouping and SIMD occupancy can change the wall time without changing the matrix element. A separately specialized compiled stage can remain faster for some selectors; fusing entire recurrence regions would instead reproduce the compiled mode's process-specific applications.

12.7 Choosing the mode

Use compiled mode when process generation is affordable and the target workload benefits from process-wide specialization. Use eager mode when many processes share one model, when generation time dominates, or when one artifact must support changing LC flow/helicity selections. The common built-in-SM JIT O3 path needs only

```
pyamplicol generate "d d~ > z g g g" artifacts/ddbar_z3g_eager \
  --model built-in-sm --execution-mode eager --color-accuracy nlc
```

External models and C++/ASM backends first require an explicit `pyamplicol model compile` prepared bundle. Missing or mismatched packs fail before DAG construction with the exact preparation command; eager generation never silently compiles a backend pack.

13 Arbitrary UFO and Serialized-JSON Models

The external-model path is a compiler, not a permissive UFO interpreter. It accepts a trusted UFO Python module or its serialized JSON representation, normalizes the model to a typed, model-independent intermediate representation, and only then exposes interactions to the current-recursion generator. A structure is executable when the compiler can construct its component expressions and prove every transformation used to lower or reuse them. Failure to prove a useful optimization merely disables that optimization; failure to prove a transformation required for correctness rejects the model with a structured diagnostic.

This section describes the implemented contract. The handwritten Standard Model in `src/pyamplicol/models/builtin/` is deliberately outside that contract: `BuiltinSMModel` may use SM-specific formulae, while `CompiledUFOModel` must derive its decisions from particle metadata, normalized tensors, propagators, couplings, and exact algebra. The boundary forbids SM PDG classifiers and built-in implementation symbols from leaking into the external-model modules.

13.1 One route from UFO and JSON to an executable model

The public entry point `compile_model_source()` in `src/pyamplicol/models/loading.py` recognizes a UFO directory, a JSON model, an already compiled model, or the special built-in model. For an external source it lazily invokes `ufo-model-loader`. Both source forms therefore enter the same route:

$$\{\text{UFO directory, serialized JSON}\} \xrightarrow{\texttt{load\_model}} \texttt{ufo-model-loader.Model} \xrightarrow{\text{compact serialization and compilation}} \texttt{CompiledModelIR}. \tag{45}$$

The UFO directory is executable Python and must therefore be trusted. The JSON form is data-only and is the portable form to archive or exchange.

The loader is first called with wrapped Lorentz indices disabled. This lets `_classify_external_propagators()` compare a model-supplied propagator against the loader's exact default Feynman-gauge candidates. Indices are then wrapped and the model is serialized with `JSONLook.COMPACT`. Consequently, UFO and JSON inputs do not maintain separate physics implementations after loading.

Every source symbol is put in a model-local namespace by `src/pyamplicol/_internal/physics/symbols.py`. A model called `sm` uses heads such as `model_sm::GC_1`, while compiler-owned currents, momenta, dummy indices, and derived couplings use the `pyamplicol::` namespace. The registry rewrites the loader's temporary `UFO::` heads; a compiled `CompiledModelIR` is rejected if an executable expression still contains a process-global `UFO::` symbol. Two loaded models can thus coexist without silently identifying equally named parameters or tensors.

The canonical IR in `src/pyamplicol/models/contracts.py` contains typed records for particles, parameters, coupling orders, propagators, vertex terms, oriented trivalent kernels, direct and closure contractions, tensor and current orderings, and Goldstone partners. It is not a bag of strings. On reload it checks the model-schema version, compiler version and fingerprint, proof identities, tensor-ordering links, and required fields before any DAG is built. The compiler fingerprint includes the pyAmpliCol, Symbolica, and `ufo-model-loader` versions and the relevant serialization and lowering policies; a stale compiled model is regenerated rather than guessed compatible.

compiler stage	resulting contract	implementation
source loading	one compact loader-model payload	`models/loading.py`: `_load_external_model()` and `preflight_model()`
parameter resolution	namespaced external and derived expressions	`models/compiler_records.py` and `ModelSymbolRegistry`
tensor normalization	canonical colour and Lorentz expressions	`models/tensors.py`, using idenso, Spenso, and Symbolica
kernel construction	oriented trivalent contracts and contact auxiliaries	`models/compiler_entry.py`, `models/compiler_kernels.py`, `models/compiler_contacts.py`
ordering construction	explicit source axes, Weyl projections, and current embeddings	`models/compiler_tensor_ordering.py`
model-owned closure	propagators, direct/closure contractions, and Goldstone policy	`models/compiler_contractions.py`, `models/compiler_gauge.py`
optimization proofs	kernel equivalence, colour-flow auxiliaries, and exact model symmetries	`models/compiler_color_flow.py` and `models/external_symmetries.py`

13.2 From a UFO vertex to concrete current kernels

A UFO vertex is a matrix over colour and Lorentz structures. The compiler expands it into separate terms

$$\mathcal{V}(1,\ldots,n) = \sum_{a,b} g_{ab}\, C_a(c_1,\ldots,c_n)\, L_b(s_1,p_1;\ldots;s_n,p_n), \tag{46}$$

and stores the original sources together with their normalized forms and the coupling-order vector of g_{ab}. The normalization in `src/pyamplicol/models/tensors.py` converts wrapped UFO heads to typed Spenso expressions. It applies idenso's `simplify_metrics`, `simplify_gamma`, and `simplify_color`; Spenso then materializes dense tensor components in a declared Cartesian order. Residual UFO tensor heads, indeterminate expressions, duplicate coordinates, or missing dense coordinates are errors.

The normalized Lorentz basis includes scalars, momenta, metrics, Dirac identity and gamma matrices, γ_5, chiral projectors, slashed momenta, and sigma tensors. The supported colour representations are the SU(3) singlet, fundamental, antifundamental, and adjoint. Exact trilinear projection recognizes four executable colour families:

$$1, \qquad \delta_i{}^{\bar{\jmath}}, \qquad (T^a)_i{}^{\bar{\jmath}}, \qquad f^{abc}. \tag{47}$$

For a proposed basis tensor B_σ, projection succeeds only when the normalized tensor obeys the componentwise identity

$$C_a(\boldsymbol{c}) = \kappa_a B_\sigma(\boldsymbol{c}) \quad \text{for every materialized colour component } \boldsymbol{c}, \tag{48}$$

with an exact scalar κ_a. Numerical projection exists as a diagnostic but does not certify an executable kernel. The symmetric adjoint invariant d^{abc}, for example, can be recognized diagnostically but is not in the current executable trilinear colour basis.

For a trilinear term, `_compile_oriented_kernels()` chooses each distinct result species and both nonidentical input orders. Spenso contracts the normalized tensor with formal left and right currents and physical momenta. The resulting component contract has the schematic form

$$K^{A_r}_{r\leftarrow ij}(J_i,J_j) = g_{ab}\,\kappa_a \sum_{A_i,A_j} L_b^{A_r}{}_{A_iA_j}(p_i,p_j,p_r) J_i^{A_i} J_j^{A_j}, \qquad p_r = -(p_i+p_j), \tag{49}$$

followed by the model-owned propagator when the result is an internal current. The coupling, perturbative orders, runtime parameters, source-leg map, colour normalization, and input/output ordering identifiers remain attached to the kernel. Weyl projection is therefore an explicit ordering/embedding contract, not an inference made later from a particle name.

The compiler recognizes a compact Yang–Mills three-vector form only after its components have been proved equal, up to one exact constant, to the dense Spenso result. This is an implementation optimization, not an alternative physics formula.

13.3 What a proof or certificate asserts

The word *proof* is used narrowly. It is a reproducible algebraic witness that an optimized or decomposed executable contract reconstructs the normalized source contract. It is not a statement that a model is physically consistent, unitary, or gauge complete. Three assurance mechanisms are distinct:

1. A **serialized decomposition proof** is attached to a four-point `CompiledVertexTerm`. Its algorithm name and version, source identity, split topology, index map, component basis, signs, and multiplicities are revalidated when the compiled model is loaded.
2. **Kernel-equivalence metadata** is written on oriented kernels. Fully lowered component expressions are compared under input exchange and an exact factor $\eta = \pm 1$. Equal kernels receive one `evaluation_class`, a representative input order, and a factor, so the DAG may evaluate the representative once and fan its value out.
3. A **model-level symmetry certificate** is re-derived by `derive_external_symmetry_certificates()` when a `CompiledUFOModel` is constructed. It binds a theorem to digests of the compiled particle, propagator, ordering, vertex, and executable-kernel contracts. It is not accepted as a free-form claim from the UFO source.

For two kernel expressions K and K', reusable evaluation requires an exact relation

$$K'(x, y; p_x, p_y) = \eta\, K(y, x; p_y, p_x), \qquad \eta \in \{-1, +1\}, \tag{50}$$

after resolving coupling aliases and canonicalizing all component expressions. For a global theorem, the digest is

$$D = \mathrm{SHA256}(\mathrm{canonicalJSON}\{\mathrm{source}, \mathrm{propagator}, \mathrm{orderings}, \mathrm{terms}, \mathrm{kernels}\})\,. \tag{51}$$

The selectors and 64-character digests must be complete and mutually consistent. Renaming a contracted tensor dummy must not change D, but a different contraction graph must. The internal helper

`_canonical_tensor_product_expressions()`

in `src/pyamplicol/models/external_symmetries.py` calls Symbolica's `Expression.canonize_tensors()` on the contracted Lorentz, colour, and spinor indices. Its reserved replacement head is

`pyamplicol::canonical_tensor_index`.

The certificate contract requires alpha-renaming invariance and contraction-graph sensitivity.

The important fail-closed distinction is:

- if a reflection or parity certificate is absent, the generator keeps both configurations and remains correct, but performs less reuse;
- if a four-point contact cannot be lowered without changing its tensor, colour, coupling, or multiplicity, generation stops rather than silently dropping or approximating that interaction.

proof class	**checked identity**	**optimization or lowering enabled**
trilinear colour projection	Eq. (48) in the supported SU(3) basis	colour-flow interpretation of an oriented kernel
kernel evaluation equivalence	Eq. (50), including coupling and ordering contracts	one evaluator value reused by symmetry-related current contributions
four-point contact decomposition	exact colour topology, open-axis Lorentz basis, orientation signs, and multiplicity	two trivalent auxiliary-current stages with no artificial pole
fundamental Fierz identity	exact T^a trilinear projection and eligible massless adjoint-vector metadata	compiler-owned singlet-subtraction auxiliary for LC flow
vectorlike parity	exact fundamental-generator colour and gamma-current Lorentz tensor	global helicity-flip representative reuse
Yang–Mills cubic/quartic sector	exact f^{abc}, three-vector tensor, quartic basis, coupling relation, and closed reachable sector	pure-adjoint helicity-zero theorem and LC trace-reflection folding
local adjoint reflection	every relevant cubic kernel is antisymmetric under left/right exchange	current reflection phase reuse
Goldstone partner contract	quantum numbers, exact mass expression, uniqueness, and propagator policy	unitary-gauge absorption or explicit model-supplied Goldstone treatment
tensor/current ordering	explicit physical axes and component permutations	stable Spenso-to-current and Weyl component maps

13.4 Four-point contacts: exact auxiliary-current factorization

Every momentum-independent valence-four term receives a `CompiledContactDecompositionProof` before lowering. The serialized algorithm is `ufo-four-point-contact-decomposition`, version 2. A proof has status `proven` or `unsupported`; its identity includes the term and vertex identifiers, particle tuple, colour and Lorentz matrix indices, original sources, and normalized expressions. A stale version or an identity mismatch is rejected.

The record family is explicit:

- `CompiledContactDecompositionProof` owns the term identity and all proved result-leg splits;
- `CompiledContactDecompositionSplit` and `CompiledContactDummyIndexMapping` describe one topology and its shared colour index;
- `CompiledContactOrientationProof` fixes every input order and sign; and
- `CompiledContactUnsupportedReason` carries a stable failure code and context.

Two decomposition kinds are implemented.

Literal colour singlet. If all four legs are colour singlets and $C = 1$, the Lorentz tensor may be split directly. A literal singlet attached to any coloured leg is not treated as an implicit contraction; it fails with `literal-singlet-with-colored-legs`.

Product of two structure constants. The coloured gauge-contact path requires exactly

$$C^{a_1a_2a_3a_4} = c\, f^{a_ia_jb} f^{ba_ka_r}, \tag{52}$$

up to exact permutations, with one and only one shared adjoint dummy b and a scalar prefactor c. For each possible result leg r, the compiler records the paired legs (i, j), remaining leg k, dummy-index map, permutation parity, LC normalization powers, and the exact scalar coefficient. It then introduces a non-propagating auxiliary current

$$A^b_{\rho\sigma}[i,j] = f^{ba_ia_j} \sum_{A_i,A_j} P_{\rho\sigma;A_iA_j} J_i^{A_i} J_j^{A_j}, \tag{53}$$

$$J_r^{A_r}\big|_{V_4} = \frac{c\, s_{ijkr}}{m_r} f^{a_rba_k} \sum_{\rho,\sigma,A_k} F^{A_r}{}_{\rho\sigma A_k} A^b_{\rho\sigma} J_k^{A_k}. \tag{54}$$

Here $s_{ijkr} = \pm 1$ is the recorded orientation parity and m_r is the number of indistinguishable assignments of the remaining input species. The original coupling and complete perturbative-order vector occur only in Eq. (54); Eq. (53) has unit coupling and no propagator. The factorization therefore introduces neither an unphysical pole nor a duplicated coupling/order weight.

The Lorentz split is not inferred from the phrase "four-gluon vertex." Spenso first contracts the chosen input pair while leaving the result and remaining-leg axes open. `_compress_contact_components()` stores a basis of distinct components and reconstructs every omitted component by an exact zero or ± 1 map. The final kernel contracts that basis and must reproduce the source tensor component by component. This same mechanism covers an SU(3) four-vector term such as the four-gluon vertex and colour-singlet electroweak-like four-vector contacts built from metric tensors. What differs is the colour proof: `two-structure-constants` for the former and `literal-color-singlet` for the latter.

The compiler deduplicates identical contact partials and fuses compatible finals after proof validation. Unsupported diagnostics distinguish an incorrect topology from a missing feature: wrong factor count, malformed structure-constant indices, non-scalar colour prefactor, non-unique shared dummy, absent result leg, and a nontrivalent structure-constant topology all have separate reason codes in `src/pyamplicol/models/compiler_contacts.py`.

13.5 Arbitrary-valence colour-singlet contact trees

Colour-singlet contacts with valence $n \geq 4$ are lowered by `_compile_color_singlet_contact_trees()` in `src/pyamplicol/models/compiler_contact_trees.py`. The already proved momentum-independent four-point case uses the preceding path; other singlet contacts are split recursively at the midpoint into a balanced binary tree. For a set of input leaves S, the generic recursion is

$$A_S = \Phi_S(A_{S_L}, A_{S_R}), \qquad S = S_L \,\dot\cup\, S_R, \qquad J_r\big|_{V_n} = \frac{g}{m_r} \Psi_r(A_{S_L}, A_{S_R}), \tag{55}$$

where the internal A_S are non-propagating, g and its coupling orders appear only in the final map Ψ_r, and m_r removes identical-input assignment multiplicity. A balanced tree has $O(n)$ auxiliary operations and $O(\log n)$ depth.

For an all-scalar contact, each auxiliary carries one component and $\Phi_S(x, y) = xy$. This is the path exercised by the arbitrary-valence scalar-contact model. For non-scalar or momentum-dependent singlet contacts, an auxiliary carries the current components and four-momentum of every physical leaf below it. At the final node, the original normalized n-point Lorentz expression is reconstructed with

$$p_r = -\sum_{i \neq r} p_i. \tag{56}$$

On Linux, very large dense contractions are evaluated in bounded stages when the direct input work exceeds 1024 components; each staged group is limited to 256 input components and its intermediate tensor is materialized before the next contraction. This changes only contraction order. The direct function `eager_color_singlet_vertex_term_components()` remains the component oracle for the tree result.

This is deliberately not advertised as an arbitrary coloured n-point algorithm. It is generic in valence, spin, momentum dependence, and Lorentz expression only after the colour tensor has been proved to be a singlet that requires no unresolved coloured internal channel.

13.6 Colour-flow and model-level symmetry proofs

The LC compiler derives its singlet-subtraction channel from the fundamental Fierz identity

$$(T^a)_i{}^{\bar{\jmath}}(T^a)_k{}^{\bar{l}} = T_R \left(\delta_i{}^{\bar{l}} \delta_k{}^{\bar{\jmath}} - \frac{1}{N_c} \delta_i{}^{\bar{\jmath}} \delta_k{}^{\bar{l}} \right). \tag{57}$$

The compiler function

`synthesize_fundamental_fierz_auxiliaries()`

in `src/pyamplicol/models/compiler_color_flow.py` is enabled only for an exactly projected T^a trilinear with fundamental fermion lines and a self-conjugate, massless adjoint vector with a standard propagator. It creates one compiler-owned colour-singlet vector auxiliary and reuses the proved Lorentz and coupling kernel. The $1/N_c$ closure coefficient is attached in one canonical orientation. No gluon name or PDG code participates.

Four further exact recognizers in `src/pyamplicol/models/external_symmetries.py` recover the reusable structure normally associated with a gauge theory:

- A **vectorlike parity kernel** has one massless adjoint vector, a fundamental/antifundamental fermion pair, exact T^a colour, and a Lorentz tensor that is a constant multiple of the gamma-current reference. Chiral-projector deformations do not pass this proof.
- A **Yang–Mills cubic kernel** has one self-conjugate massless adjoint-vector species, a standard propagator, exact f^{abc}, and the three-vector Lorentz tensor up to an exact constant. The effective cubic coupling is retained symbolically.
- A **Yang–Mills quartic group** must uniquely span the three $ff \times gg$ basis elements of the four-vector vertex, and each coefficient must obey the exact relation to the square of the cubic coupling. Recognizing three individually familiar-looking terms is insufficient if the coupling relation fails.
- A **local adjoint-current reflection** requires every relevant cubic kernel to satisfy Eq. (50) with $\eta = -1$, exact f^{abc}, and a standard propagator.

A global Yang–Mills theorem is emitted only if the reachable trivalent compiled sector is closed: starting from the candidate adjoint source, every kernel reachable from two already reachable currents must itself be certified Yang–Mills. Contact auxiliaries are included in this traversal. One extra reachable non-Yang–Mills interaction therefore disables the global theorem rather than being ignored. A complete certificate enables the pure-adjoint helicity-zero theorem, global helicity-flip representatives, LC trace reflection, and shared single-trace basis. The ordinary DAG remains complete when the certificate is absent.

These generic proofs recover the built-in-SM current topology for the established UFO-SM validation matrix without assuming that every external model must do so. Validation covers electroweak, QCD, top, pure-gluon, charged-current, multi-boson, and up to three-quark-line examples. Equivalent built-in and UFO-SM models must have

equal minimal coupling limits and equal current, interaction, amplitude-root, and colour-sector counts; exact kernel reuse may reduce evaluator calls but may not increase them. The same proof is invariant under arbitrary particle-name, PDG, and inventory-order changes. This establishes the certified classes, not a universal topology theorem for all UFO syntax.

13.7 Propagators, closures, Goldstones, and spin two

Propagators are model contracts rather than hard-coded particle identities. For standard scalar, Dirac, vector, and spin-two records, the evaluator in `src/pyamplicol/models/external_evaluation.py` implements the expected runtime-mass and runtime-width kernels, schematically

$$P_0(p) = \frac{i}{p^2 - m^2 + im\Gamma}, \qquad P_{1/2}(p) = \frac{i(\gamma^\mu p_\mu + m)}{p^2 - m^2 + im\Gamma}, \tag{58}$$

$$P_1^{\mu\nu}(p) = \frac{-i}{p^2 - m^2 + im\Gamma}\left(g^{\mu\nu} - \frac{p^\mu p^\nu}{m^2}\right), \qquad P_2^{\mu\nu,\rho\sigma}(p) = \frac{i\,\Pi^{\mu\nu,\rho\sigma}(p)}{p^2 - m^2 + im\Gamma}. \tag{59}$$

The massless-vector branch is Feynman gauge and does not use the singular $p^\mu p^\nu/m^2$ term. Weyl currents use explicit chirality projections. For spin two, Π is the de Donder massless or Fierz–Pauli massive projector represented in a 16-component ordered tensor space. Synthetic contact auxiliaries have the identity map and no propagator; the Fierz singlet-subtraction auxiliary propagates as its compiler-owned massless vector.

A custom UFO propagator is accepted when its normalized numerator and denominator can be materialized through the same Spenso ordering contract. It is applied exactly and is not silently converted to a preferred gauge. Its presence intentionally disables symmetry theorems that require the standard propagator.

`compile_contraction_records()` emits explicit scalar, Dirac, opposite Weyl-chirality, mostly-minus vector, and rank-two tensor-product-metric contractions. Rusticol consumes these records and does not infer a closure from component count. Goldstone matching in `compile_goldstone_partner_records()` similarly uses quantum-number flow and the exact resolved mass expression. A unique massive vector with a default propagator absorbs its Goldstone in the unitary-gauge path; a custom vector propagator retains the model-supplied Goldstone. Ambiguous matches fail closed.

UFO spin code 5 is therefore a real current type, not a special-case scalar: it has two ordered Lorentz axes, 16 components, tensor-product closure, and a spin-two propagator. The packaged massless scalar-gravity model validates the generic tensor route, including axis relabelling and transposed source storage. Massive spin two is accepted with an explicit experimental warning; arbitrary massive spin-two phenomenology is not yet a qualified release capability.

13.8 Implemented support boundary

The support matrix below separates implemented current capabilities from unsupported classes. "Proof-gated" means that qualifying models work without an SM-specific exception; it does not mean every algebraically equivalent UFO spelling is already recognized.

area	class	status	exact boundary
source	UFO directory / JSON	supported	both lower through one loader-model payload; UFO Python must be trusted
spin	scalar, Dirac, vector	supported	standard wavefunctions, propagators, current orderings, and closures
spin	massless spin two	supported	16-component tensor path tested by the packaged scalar-gravity model
spin	massive spin two	experimental	Fierz–Pauli path exists, but broad model validation is incomplete
spin	UFO ghosts	partial	one-component metadata and ordering exist; propagating ghosts require an exactly lowerable custom propagator and are not broadly qualified
spin	Majorana/FNV, spin 3/2	unsupported	no fermion-number-violating flow or Rarita–Schwinger current contract
colour	$1, 3, \bar{3}, 8$ of SU(3)	supported	LC and current contracted NLC/full basis

area	class	status	exact boundary
colour	$6, \bar{6}$, extra groups	unsupported	no sextet or multi-group flow/contraction representation
arity 3	$1, \delta, T, f$ colour	supported	exact projection plus generic materializable Lorentz components
arity 3	d^{abc}, ϵ, K_6, T_6	unsupported	recognized or known source heads are not sufficient to construct an executable colour contract
arity 4	colour singlet	supported	momentum-independent proof path; generic singlet-tree path for other materializable tensors
arity 4	two-f gauge colour	proof-gated	one shared adjoint dummy, trivalent topology, scalar prefactor, exact Lorentz reconstruction
arity 4	other coloured tensors	unsupported	no certified auxiliary colour channel
arity > 4	colour-singlet contact	supported	balanced no-pole tree; original coupling/order only at final node
arity > 4	coloured contact	unsupported	no general irreducible-channel factorization proof
Lorentz	idenso/Spenso basis	supported	all source tensor heads must normalize and materialize with explicit axes
Lorentz	form factors / unknown functions	unsupported	no executable and serializable function contract
propagator	standard $0, \frac{1}{2}, 1, 2$	supported	runtime mass/width and explicit ordering/closure
propagator	custom UFO tensor	proof-gated	exact Spenso lowering; gauge conversion and affected symmetry reuse are disabled
symmetry	vectorlike/Yang–Mills/reflection	proof-gated	exact normalized identities and complete reachable-sector digest

Preflight in `preflight_model()` rejects unsupported spin or colour codes, Majorana fermions, form factors, wrong known-function arities, and unknown expression functions before expensive compilation. Tensor normalization and contact lowering impose the narrower executable boundary after preflight. Thus recognizing a UFO head such as `K6` during syntax inspection does not imply that a sextet colour current can be built.

13.9 Support boundary and extension requirements

Support is defined by algebraic contracts rather than process-family or PDG exceptions. A new class requires a typed source representation and normalization, an explicit component ordering and current embedding, a direct contraction and colour-flow interpretation, and an exact proof of orientation signs, coupling placement, multiplicities, and any evaluator reuse. The proof must remain invariant under dummy-index, source-label, and particle-name renaming, and its resolved sum must agree with a direct or higher-precision oracle. Without those contracts the model is rejected at the corresponding boundary rather than assigned an inferred implementation.

unsupported or restricted class	required contract
additional scalar UFO functions	define a serializable Symbolica lowering and derivatives/branch semantics where relevant
adjoint d^{abc} trilinears	add a physical colour basis, projection, contraction, and LC/NLC/full interpretation
epsilon and sextet tensors	add sextet current identities, conjugation, flow basis, and sparse colour metric before tensor lowering
other coloured four-point contacts	prove a complete set of auxiliary irreducible channels and exact reconstruction, including repeated representations and multiplicities
coloured valence above four	general representation-channel tree with associativity/basis-change proofs and controlled intermediate growth
Majorana/FNV fermions	fermion-flow orientation, charge conjugation, signs, wavefunctions, propagators, and closure contracts
spin 3/2	ordered Rarita–Schwinger wavefunctions, projectors, contractions, and tensor-component qualification
multiple colour groups	group-qualified representations, independent invariant tensors, product flow bases, and contracted colour metrics
general massive spin two	broader model and gauge/constraint validation around the existing 16-component propagator contract

Validation includes successful examples and deliberately deformed tensors, couplings, propagators, index orderings, and proof records. This negative coverage is part of the advertised support boundary: an unfamiliar model either satisfies the same explicit algebraic contract or receives a diagnostic where a new proof is required.